AEGIS-NET

Technical Handbook

Network intelligence, AI root-cause analysis and a policy-gated DevSecOps agent: from first principles to implementation details

A learning guide, architecture reference and project handover for the Aegis-Net prototype.

Version	1.0
Date	2026-10-20
Repository state	commit 5d742e7 on branch master (working tree includes the documentation under deliverables/)
Author field	Aegis-Net project documentation (generated with Claude Code from the repository); no individual or organisation is named as author
Data notice	All data in Aegis-Net is synthetic. No claim is made about real networks.

Contents

1 Introduction to Aegis-Net 7
1.1 What Aegis-Net is 7
1.2 The problem it addresses 7
1.3 Who would use it, and where it sits 7
1.4 Capabilities and boundaries 8
1.5 High-level workflow 9
2 Foundations for a Beginner 10
2.1 What a telecommunications network does 10
2.2 Call-detail records (CDRs) 10
2.3 KPIs, counters and the metrics Aegis-Net tracks 10
2.4 Anomalies, incidents and root causes 11
2.5 APIs, databases, containers, deployments 11
2.6 What an LLM does, and what an agent adds 11
3 System Architecture 12
3.1 Components and why each exists 12
3.2 Why more than one language 13
3.3 Trust boundaries 13
4 The Complete Data Flow 14
5 The C Call-Analysis Library 16
5.1 Public API and header design 16
5.2 Structures 16
5.3 Parsing, validation, strict and lenient modes 16
5.4 Status-code classification and rate formulas 17
5.5 Aggregations 17
5.6 Call anomaly detection 17
5.7 Memory allocation and ownership 19
5.8 Build and test layers 19
6 Python and C Integration 20
6.1 ctypes from first principles 20
6.2 Structure layout and the ABI 20
6.3 Function prototypes and pointers 20
6.4 Native allocation, freeing and exception mapping 21
6.5 Library discovery 21
6.6 Why integer width and overflow validation matter 21
6.7 Why Python orchestrates and C computes 22
7 Synthetic Network Telemetry 23
7.1 What it produces 23

7.2 Baselines, load cycle, noise and determinism ... 23
7.3 The causal model ... 23
7.4 Incident scenarios ... 24
7.5 The seeded memory-leak and deployment incident ... 24
8 Anomaly Detection, From Basic to Advanced ... 27
8.1 Worked example used throughout ... 27
8.2 Moving average and rolling standard deviation ... 28
8.3 Z-score ... 28
8.4 EWMA ... 28
8.5 Absolute thresholds ... 28
8.6 Rate of change ... 28
8.7 Scoring, severity, episodes and baselines ... 29
8.8 Detectors versus the simulator ... 30
8.9 Strengths, failure modes and limits across all detectors ... 30
9 Correlation and Root-Cause Reasoning ... 31
9.1 Why symptoms share a cause ... 31
9.2 What the engine actually computes ... 31
9.3 The seeded evidence chain ... 32
10 Database and Backend API ... 35
10.1 Relational modelling in this project ... 35
10.2 SQLAlchemy, Alembic and PostgreSQL ... 36
10.3 FastAPI, Pydantic and validation ... 36
10.4 Endpoint reference ... 37
10.5 Pipeline execution ... 38
11 Authentication, Security, and Auditability ... 39
11.1 Authentication versus authorisation ... 39
11.2 Approval gates ... 40
11.3 Request identifiers, logging and audit ... 40
11.4 Input validation, limits and rate limiting ... 41
11.5 Prompt injection and untrusted log content ... 41
11.6 Safe command execution and secrets ... 41
12 LLM Provider Abstraction ... 42
12.1 Basics ... 42
12.2 Provider abstraction ... 42
12.3 Configuration ... 42
12.4 Sequence of an investigation ... 43
12.5 Deterministic testing with the mock ... 43
13 Root-Cause Analysis Engine ... 44
13.1 Workflow ... 44

13.2 Facts, inferences, hypotheses, recommendations 44
13.3 Worked example from the seeded incident 45
13.4 Presenting conclusions without exposing hidden reasoning 45
14 Report Generation 46
14.1 Data model and numeric consistency 46
14.2 Narrative and template fallback 46
14.3 Renderers and escaping 46
14.4 An actual excerpt 46
15 Autonomous DevSecOps Agent 48
15.1 The loop 48
15.2 Tools and risk classes 48
15.3 Policy and approval 49
15.4 The sandbox and the disposable workspace 49
15.5 Synthetic advisories 50
15.6 The three scenarios 50
15.7 What is real, simulated, approved and blocked 51
16 Frontend and Dashboard 52
16.1 Structure 52
16.2 Authentication handling 52
16.3 Pages and how users interact 52
17 Docker and Deployment 54
17.1 Concepts 54
17.2 The images 54
17.3 Services, ports, volumes 55
17.4 Commands 55
17.5 Troubleshooting 55
18 Testing and Quality Assurance 56
19 How to Run and Demonstrate Aegis-Net 57
19.1 Prerequisites and repository map 57
19.2 Environment 57
19.3 Offline demo (no key, no Docker) 57
19.4 Docker with the dashboard 57
19.5 A guided walk-through 57
19.6 Tests 58
19.7 Live providers 58
19.8 Common failures 58
20 Limitations, Trade-offs, and Future Work 59
20.1 Honest assessment 59
20.2 Prioritised roadmap (recommendations, not existing features) 59

21 Advanced Technical Deep Dives 60
21.1 C and Python ABI and memory ownership 60
21.2 Statistical detector design and trade-offs 60
21.3 Evidence provenance and hypothesis evaluation 60
21.4 Structured LLM tool execution 60
21.5 Policy enforcement and threat modelling 60
21.6 Database integrity 60
21.7 Audit-chain verification 60
21.8 Integration testing 61
21.9 Observability 61
21.10 Performance and scaling considerations 61
22 Glossary and Reference Appendix 62
22.1 Glossary 62
22.2 Important formulas 63
22.3 Repository map 63
22.4 Component and interface reference 63
22.5 Environment variables 64
22.6 Terminal commands 64
22.7 Most useful source files and tests 65
Appendix A: Screenshot Inventory 66
Appendix B: Verification Record 66
Appendix C: References 67

List of figures

Figure 1. High-level workflow from synthetic data to audited remediation 9
Figure 2. Aegis-Net system architecture, layered 12
Figure 3. End-to-end sequence of API calls for one incident 14
Figure 4. Call Analytics page: totals, drop rate by cell, status codes and C-library anomalies 18
Figure 5. C to Python call, ownership and release 21
Figure 6. The simulator's causal chain 23
Figure 7. Seeded run, affected cell versus an unaffected control cell 25
Figure 8. Network Overview page for the same incident 25
Figure 9. Detector processing pipeline 27
Figure 10. The worked example, with baseline, threshold and EWMA 27
Figure 11. Score as a function of z, with severity bands 29
Figure 12. Anomalies page filtered to critical severity 29
Figure 13. Evidence in, ranked hypotheses out 31
Figure 14. The timeline the engine builds 33
Figure 15. Why deployment_regression outranks memory_exhaustion 33
Figure 16. Incidents list: one critical incident with its leading hypothesis 34
Figure 17. Entity-relationship diagram of the 19 tables (from `app/models/`) 36
Figure 18. Dashboard behaviour when a viewer opens the Audit page 39
Figure 19. Separation of duties message in the agent panel 39
Figure 20. Audit Log page: hash chain valid (38 rows) 40
Figure 21. RCA tool-calling sequence 43
Figure 22. Root Cause Analysis page for INC-2026-001 (mock provider) 45
Figure 23. AI Reports page with the executive summary and KPI table 47
Figure 24. Agent lifecycle 48
Figure 25. Risk classification and approval workflow 49
Figure 26. DevSecOps Agent page: one closed run and two runs awaiting approval 50
Figure 27. Security page with the synthetic finding 51
Figure 28. Dashboard 52
Figure 29. Dashboard navigation and workflows 53
Figure 30. Container topology 54

1 Introduction to Aegis-Net

> **Implementation status.** Status vocabulary used in this handbook. VERIFIED means the behaviour is implemented and a test or recorded run in this repository exercised it. UNVERIFIED HERE means the code exists but was not exercised in the relevant environment. SIMULATED means the behaviour is simulated or for demonstration only. NOT IMPLEMENTED means not implemented or deferred. Planning documents were not accepted as proof: each claim was checked against source code, tests, or a command run for this handbook (Appendix B).

1.1 What Aegis-Net is

Imagine you run a city's bus network. One morning passengers complain that buses are late on four routes. A dispatcher looks at a map, a spreadsheet of delays, and a list of changes made overnight: a new software release for the ticket machines, a road closure, a staff roster change. Finding which change caused the delays means gathering evidence from several places, ordering events in time, and ruling out suspects. Then someone has to decide what to do, and the decision should be recorded.

Aegis-Net does the same job for a mobile telecommunications network, in software. It is a prototype that:

- reads call-detail records (CDRs) with a C library and computes call statistics and outliers;
- watches network key performance indicators (KPIs) such as latency and packet loss for anomalies;
- correlates anomalies with deployments, configuration changes and security events into an incident;
- runs an AI-assisted root-cause analysis (RCA) that can only use read-only tools;
- writes an incident report; and
- runs a policy-gated DevSecOps agent that proposes fixes, waits for human approval where required, verifies the result, and records everything in a tamper-evident audit log.

> **Warning.** All data in Aegis-Net is synthetic. The telemetry, call logs, deployments, security events and DEMO-CVE advisories are generated by this repository. No real network, customer or vulnerability is described, and the production rollback is a simulation. SIMULATED

1.2 The problem it addresses

Network operators run thousands of cells (radio sites) served by gateways and core software. When service degrades, three things make investigation slow:

1. **Evidence is scattered.** Metrics live in a monitoring system, deployment records in a CI/CD tool, security alerts in an intrusion detection system (IDS), and logs on servers.
2. **Symptoms arrive in a cascade.** A single fault shows up as a dozen alerts, not one. The first symptom is often not the one people notice.
3. **Decisions need an audit trail.** Rolling back a production release is risky and must be justified and approved.

Manual investigation typically means: confirm the problem, find the affected elements, build a timeline, list recent changes, compare symptom order with how the system should behave, form a hypothesis, test it, act, and verify recovery. Aegis-Net automates the evidence gathering, ordering and bookkeeping. It does not automate judgement: confidence is capped, every cause is labelled a hypothesis, and risky actions need a human.

1.3 Who would use it, and where it sits

Discipline	What it cares about	How Aegis-Net relates
Network operations (NOC)	Is the network healthy, what is broken, how severe	Dashboard, incidents, KPI charts

Discipline	What it cares about	How Aegis-Net relates
Site reliability engineering (SRE)	Service levels, incident response, postmortems	Incident timeline, RCA, reports
AIOps (AI for IT operations)	Using data and models to reduce operator toil	Anomaly detection, correlation, tool-using RCA
DevSecOps	Safe, secure change delivery	Agent scenarios: vulnerable dependency, failed CI, rollback

The intended readers of this handbook are a newcomer, a student, an interviewer evaluating the design, and a team reviewing technical decisions.

1.4 Capabilities and boundaries

Table 1. Capability summary with implementation status

Capability	Status	Evidence
C call-analysis library with strict warnings, sanitizers, fault injection	VERIFIED	`make c-test`, `make c-test-asan` run for this handbook: 2 of 2 tests passed in each
Python ctypes binding with ABI checks	VERIFIED	`backend/tests/test_binding_abi_and_ownership.py` (28 tests) passed
Deterministic telemetry simulator, 17 KPIs, 9 incident types	SIMULATED VERIFIED	`test_simulation.py` (31 tests); data is synthetic by design
Four interpretable anomaly detectors	VERIFIED	`test_analytics.py` (24 tests)
Correlation into incidents with ranked hypotheses	VERIFIED	`test_correlation.py` (29 tests); seeded run reproduced here
RCA engine, nine read-only tools, mock provider	VERIFIED	`test_rca.py` (45 collected, 44 run)
Anthropic, OpenAI, local LLM providers	UNVERIFIED HERE	Request shapes tested with fake clients; never called live (no key)
Reports in JSON, Markdown, HTML	VERIFIED	`test_reporting.py` (11 tests)
Report in PDF	NOT IMPLEMENTED	Not implemented
DevSecOps agent, policy engine, sandbox	VERIFIED	`test_agent.py` (36 tests)
Production rollback	SIMULATED	Re-simulates telemetry; no real infrastructure is touched
REST API, token RBAC, hash-chained audit	VERIFIED	`test_api_core.py`, `test_security.py`
React dashboard	VERIFIED	`make frontend-test` run for this handbook (Vitest 10 of 10, build ok); browser check recorded; screenshots supplied
Docker Compose stack	VERIFIED	Recorded smoke test in `IMPLEMENTATION_PLAN.md`; not re-run for this handbook
Isolation Forest, DBSCAN, OpenTelemetry, Grafana	NOT IMPLEMENTED	Optional in the specification; not built
Behaviour on real telemetry	NOT IMPLEMENTED	Never tested

> **Note.** Aegis-Net is a working prototype. It is not production-ready: there is no TLS termination, no external identity provider, no secret manager and no high availability (Chapter 20).

1.5 High-level workflow



Figure 1. High-level workflow from synthetic data to audited remediation

Figure 1 shows the ten stages. Each stage reads the state left by the previous one; after stage 5 the evidence lives in the database, so the RCA engine, report builder and agent all work from stored facts rather than from in-memory objects. Chapter 4 follows one incident through every stage.

2 Foundations for a Beginner

> **Note.** This chapter explains background concepts in plain language. It is general background, not a description of Aegis-Net features, except where it says "in Aegis-Net".

2.1 What a telecommunications network does

A mobile network connects phones to each other and to the internet. A phone talks by radio to a **cell** (a radio site, sometimes called a base station). Cells connect through a backhaul network to **gateway** software in the core network, which routes traffic onward. In Aegis-Net the synthetic topology has 12 cells (`CELL-101` to `CELL-112`) served by three packet-gateway nodes (`pgw-1` to `pgw-3`), four cells each. Nothing about this topology is real; it is chosen to make shared-node failures easy to see.

2.2 Call-detail records (CDRs)

A **call-detail record** is one row describing one call attempt: when it happened, who called whom, which cell carried it, how long it lasted, how it ended, and some radio quality numbers. Aegis-Net uses eight columns:

```
timestamp,caller,callee,cell_id,duration,status_code,signal_strength,packet_loss
2026-03-01T14:05:00Z,+15550001,+15550002,CELL-102,184,200,-82,1
```

Operators analyse millions of such rows. Counting how many calls dropped in each cell, and spotting a cell that is much worse than its neighbours, is a standard first diagnostic.

2.3 KPIs, counters and the metrics Aegis-Net tracks

A **key performance indicator (KPI)** is a number summarising service quality over a short interval. Aegis-Net samples 17 KPIs per cell every minute (units from `backend/app/simulation/metrics.py`):

Table 2. The 17 simulated KPIs

KPI	Unit	Bad direction	Plain meaning
latency_ms	ms	up	Round-trip delay. If you shout and hear the echo 22 ms later, latency is 22 ms
jitter_ms	ms	up	How much the delay varies from packet to packet
packet_loss_pct	%	up	Share of data packets that never arrive: 3 lost per 100 is 3 %
throughput_mbps	Mbit/s	down	Delivered data rate
retransmission_rate	%	up	Share of data sent again because it was lost
rssi_dbm, rsrp_dbm	dBm	down	Received signal strength (RSSI) and reference signal power (RSRP); closer to 0 is stronger, so -80 dBm is better than -110 dBm
rsrq_db, sinr_db	dB	down	Signal quality and signal to interference-plus-noise ratio (SINR)
handover_success_rate / failure_rate	%	down / up	Whether a phone moving between cells keeps its call
call_setup_success_rate	%	down	Share of call attempts that start
call_drop_rate	%	up	Share of established calls that end abnormally
cpu_utilization, memory_utilization	%	up	Load on the serving gateway: 57 % memory means 57 of every 100 units are in use
active_users	users	both	Connected users
bandwidth_utilization	%	up	Backhaul usage

"Bad direction" matters: a rise in latency is bad, a fall in throughput is bad, and users can be unusual in either direction.

2.4 Anomalies, incidents and root causes

An **anomaly** is a measurement that is unusual compared with what is normal for that series, such as memory at 100 % when it normally sits near 57 %. An **incident** is an operational event that needs attention, built from several related anomalies. **Root-cause analysis (RCA)** asks which underlying change explains the symptoms. A symptom (high latency) is not a cause (a software release that leaks memory, which raises CPU, which raises latency).

2.5 APIs, databases, containers, deployments

- An **application programming interface (API)** is how programs talk to each other. Aegis-Net exposes an HTTP REST API.
- A **database** stores structured records durably. Aegis-Net uses PostgreSQL in Docker and SQLite for the offline demo.
- A **container** packages software with its dependencies so it runs the same everywhere. **Docker Compose** starts several containers together.
- A **deployment** is a new software version going live on a server. Deployments are a prime suspect in incidents because they are deliberate changes.

2.6 What an LLM does, and what an agent adds

A **large language model (LLM)** predicts text. Given a question and descriptions of available tools, it can answer with a request such as "call get_anomalies with these arguments". It cannot, by itself, do anything; a program must run the tool and give the result back. An **agent** is a program that wraps repeated cycles of observe, decide, act. In Aegis-Net the LLM only helps the RCA step (and writes one summary paragraph in reports). The DevSecOps agent's plan is deterministic and rule-based, which is a deliberate safety choice (Chapter 15).

> **Example.** A worked mini-example. Suppose a cell's latency has hovered around 22 ms with a standard deviation of 1 ms. A reading of 250 ms is 228 standard deviations away. No judgement is needed to call that unusual. A reading of 24 ms is two standard deviations away and could easily be noise. Chapter 8 turns this intuition into algorithms.

3 System Architecture

3.1 Components and why each exists

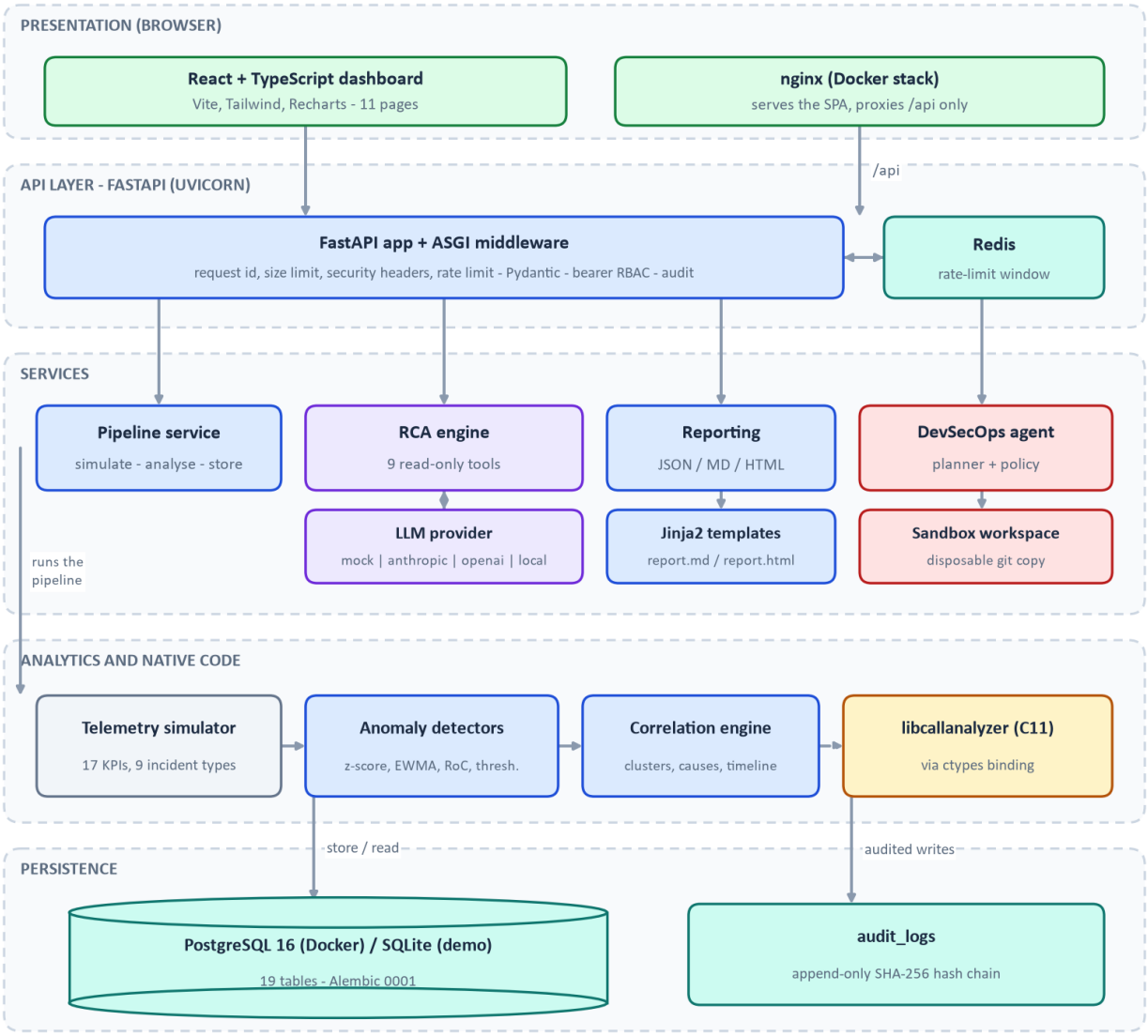


Figure 2. Aegis-Net system architecture, layered

Figure 2 groups the components into layers. Colour encodes kind: amber is native C, blue is Python backend, purple is AI, red is policy and security, teal is storage, green is the user interface, grey is synthetic data.

Table 3. Components, location, responsibility and rationale

Component	Location	Responsibility	Why it exists
C call-analysis library	`c-library/`	Parse and validate CDR CSV, aggregate, detect call outliers	CDR files are large and untrusted; a small allocation-disciplined native library gives bounded parsing and speed behind a stable ABI
ctypes binding	`backend/call_analyzer/`	Load the shared library, marshal structs to Python dicts, map errors to exceptions	Lets Python orchestrate without re-implementing the computations
Call-log generator	`scripts/generate_call_logs.py`	Seeded synthetic CDRs with an optional progressive cell fault	Reproducible test input with no dependency on the backend

Component	Location	Responsibility	Why it exists
Telemetry simulator	`backend/app/simulation/`	Causal 17-KPI model, 9 incident injectors, events, deployments, rollback	A controllable test instrument with known ground truth
Anomaly detection	`backend/app/analytics/detectors.py`, `engine.py`, `config.py`	Four detectors, scoring, episodes	Interpretable detection whose every decision can be explained
Correlation engine	`backend/app/analytics/correlation.py`, `causal_graph.py`	Cluster symptoms, score candidate causes, build timeline	Turns many alerts into one explained incident
Database	`backend/app/models/`, `backend/alembic/`	19 tables, JSON for evidence documents	Durable shared state for services and the dashboard
FastAPI and Pydantic	`backend/app/api/`, `schemas/`	26 routes, validation, OpenAPI	A typed, documented boundary
Auth, RBAC, audit	`backend/app/security/`	Hashed tokens, roles, hash-chained log	Accountability for every mutation
LLM abstraction	`backend/app/llm/`	One interface over mock, Anthropic, OpenAI, local	Swappable providers; offline default
RCA engine and tools	`backend/app/rca/`	Bounded tool loop, schema-validated answer	Evidence-cited conclusions
Reporting	`backend/app/reporting/`	Report document, Markdown/HTML templates	Consistent, escaped outputs
DevSecOps agent	`backend/app/agents/`	Playbooks, policy, sandboxed runner, 20 tools	Controlled remediation with approvals
Dashboard	`frontend/`	React, TypeScript, Vite, Tailwind, Recharts	Operator view on the real API
Docker stack	`docker-compose.yml`, Dockerfiles	postgres, redis, backend, frontend (nginx)	One-command local deployment

3.2 Why more than one language

- **C** for the hot, untrusted-input path: CSV parsing, grouped aggregation, robust statistics. Memory is managed explicitly, which is both the risk and the reason it needs sanitizer and fault-injection testing (Chapter 5).
- **Python** for orchestration, analytics, the API and the agent. NumPy handles series maths; SQLAlchemy and FastAPI handle persistence and HTTP.
- **TypeScript** for the dashboard so the API's JSON shapes are checked at compile time.
- **SQL** (via PostgreSQL) for relational integrity.

The languages meet at three explicit interfaces: the C header (`call_analyzer.h`), the REST API with its OpenAPI schema, and the database schema. CLAUDE.md requires such interfaces to be kept explicit.

3.3 Trust boundaries

Four boundaries drive the security design: CSV into the C parser (untrusted, bounded); LLM output into the RCA engine and agent (untrusted, schema-validated); HTTP clients into the API (authenticated, authorised, rate-limited, size-limited); and the agent into the operating system (fixed executables, argument lists, confined directory, scrubbed environment).

4 The Complete Data Flow

This chapter follows INC-2026-001, the seeded incident, through every stage. Numbers come from `make demo` as run for this handbook (seed 7, exit status 0) unless stated.

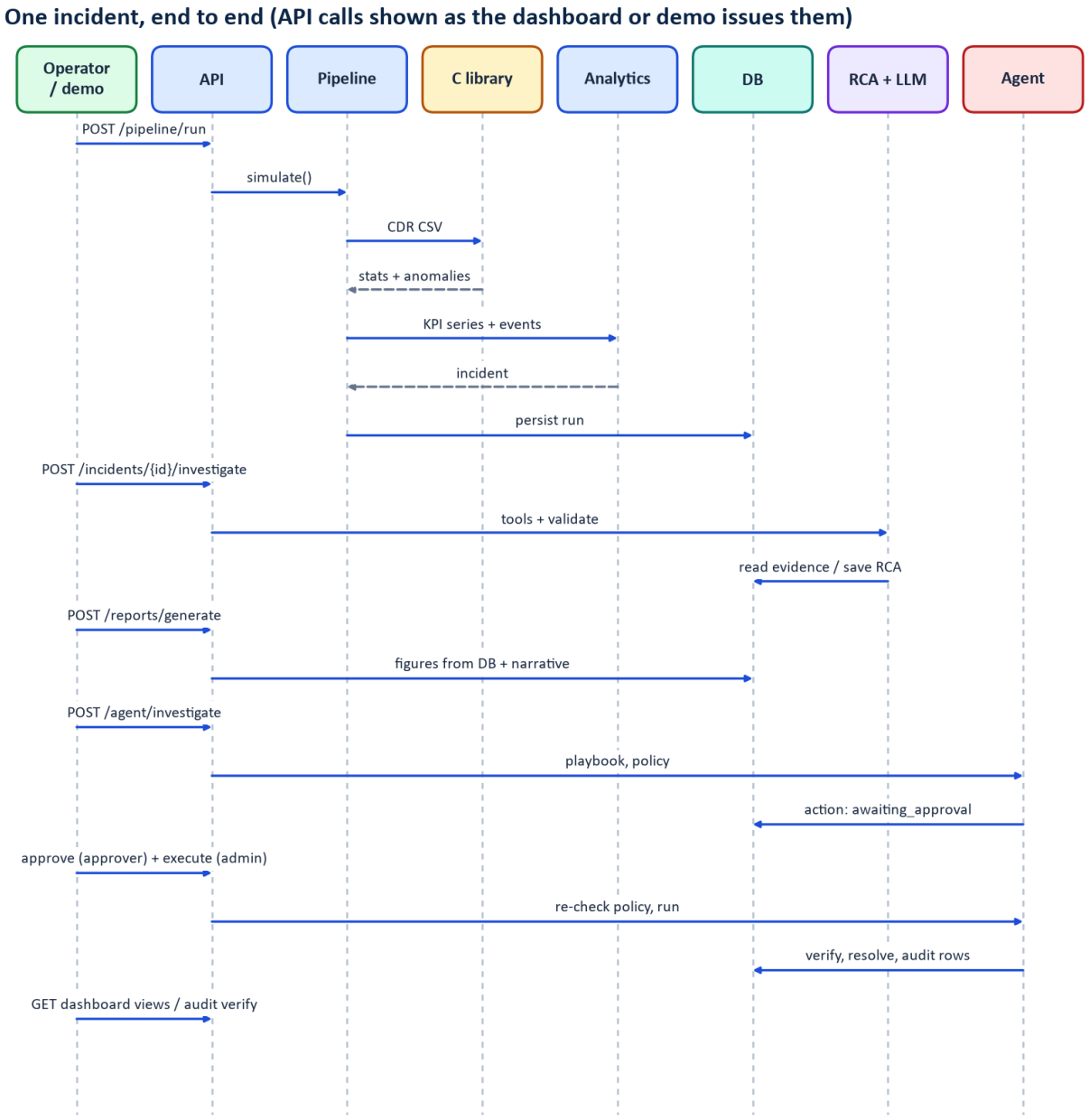


Figure 3. End-to-end sequence of API calls for one incident

Table 4. Step-by-step walk-through

#	Responsible component	Input	Processing	Output	Important error cases
1	`simulation.simulate`	`Scenario` (seed 7, 360 min, memory-leak deployment at 14:05 on pgw-1)	Causal model with AR(1) noise; injectors alter drivers only	17 KPIs x 12 cells per minute, events, deployment records	Invalid scenario raises `ValueError` (for example rollback without a deployment-induced incident)
2	`call_analyzer` + C library	CDR CSV (21,400 calls)	Parse, validate, aggregate, robust outliers	Statistics and 8 call anomalies	Parse errors, limits, out-of-memory map to typed exceptions

#	Responsible component	Input	Processing	Output	Important error cases
3	`analytics.detect_anomalies`	KPI series	Four detectors per series, episode grouping	36 anomaly episodes	Unknown metric raises `KeyError`; non-increasing epochs raise `ValueError`
4	`analytics.correlate`	Anomalies, events, call anomalies	Cluster, gate, score causes	One CRITICAL incident, leading cause `deployment_regression` 0.8947	No incident if fewer than two metrics or no symptom of severity high or above
5	`services.pipeline`	All of the above	Run-scoped ids; bulk inserts	Rows in 19 tables	Failures roll back; run status recorded
6	`rca.engine.investigate`	Incident id	Tool loop, validation	Completed RCA (85 %, 12 tool calls with the mock) or `failed`	Provider error, bad tool arguments, invalid or uncited answer: reported, never invented
7	`reporting.build_report`	Incident, RCA	Figures from the database; one narrative paragraph	Report in three formats	Ungrounded narrative numbers rejected; provider errors fall back to a template
8	`agents.service`	Scenario `production_anomaly`	Playbook; policy evaluation per action	Proposed `rollback_deployment` (HIGH_RISK) awaiting approval	Policy denies when the high-risk flag is off
9	`agents.service.execute_action`	Approved action	Policy re-checked; simulated rollback; recovery check	`recovered=True`, incident resolved	Execute before approval returns HTTP 409; wrong role returns 403
10	`security.audit`, dashboard	Every action	SHA-256 chain; read views	Audit chain valid (24 rows in the demo)	Tampering detected by `verify_chain`

Note. Step 8 and 9 in the screenshot of Chapter 16 differ from the demo: the screenshot environment left high-risk actions disabled, so the rollback shows as blocked and the incident stays open. The demo run enables `AGENT_ALLOW_HIGH_RISK` for itself.

The detection chain observed in the demo run was: memory at 14:13, CPU at 14:31, latency at 14:40, jitter at 14:45, packet loss at 14:50, retransmissions at 14:52, call drop and throughput at 14:53. After the simulated rollback, the last 30 minutes of KPIs were within the practical deltas of the baseline: memory 57.832 (baseline 57.148), latency 23.216 ms (22.331), drop rate 0.771 % (0.773).

5 The C Call-Analysis Library

> **Implementation status.** VERIFIED Built with `-Wall -Wextra -Wpedantic` plus 13 further warning flags and `-Werror` (`c-library/CMakeLists.txt`). `make c-test` and `make c-test-asan` were run for this handbook: both reported "100% tests passed, 0 tests failed out of 2" (a normal and an allocation-failure-injection executable). The same test source holds 28 test functions.

5.1 Public API and header design

The header `c-library/include/call_analyzer.h` is the authoritative contract. Its conventions are worth stating because every other layer relies on them:

- Functions returning `int` return `CA_OK` (0) or a negative `CaStatus`: `CA_ERR_NULL_ARG` (-1), `CA_ERR_INVALID_ARG` (-2), `CA_ERR_IO` (-3), `CA_ERR_NOMEM` (-4), `CA_ERR_PARSE` (-5), `CA_ERR_OVERFLOW` (-6), `CA_ERR_LIMIT` (-7).
- Arrays returned through out-parameters are allocated by the library and released only with the matching `free_*` function. On failure the output pointer is set to NULL and the count to 0.
- Fixed-size string fields (`char[32]`) are always NUL-terminated; inputs that do not fit are rejected, never truncated.
- There is no global mutable state in the production build, so calls are re-entrant.
- Only the public API is exported: the shared library is built with hidden visibility and `CA_API` marks the exported functions.

The main entry points are `parse_call_log_ex`, `parse_call_log_buffer`, `calculate_statistics`, `calculate_cell_statistics`, `calculate_failure_code_statistics`, `calculate_time_window_statistics`, `calculate_caller_statistics` and `detect_call_anomalies`.

5.2 Structures

`CallRecord` holds four 32-byte strings, four `int` fields (duration, status code, signal strength, packet loss) and an `int64_t epoch_seconds`. Statistics structures (`CallStatistics`, `CellStatistics`, `TimeWindowStatistics`, `CallerStatistics`, `FailureCodeStatistics`) hold counts as `size_t` and rates as `double`. `CaStructSizes` exports `sizeof` of every structure so the Python binding can verify its mirrors at load time (Chapter 6).

5.3 Parsing, validation, strict and lenient modes

The parser (`src/parser.c`) reads line by line, either from a file or from a memory buffer, and is deliberately strict about what it accepts.

- **Header detection.** If the first non-blank row's first field equals `timestamp` (case-insensitive), the row is treated as a header and all eight column names must match in order; otherwise the file is headerless. A UTF-8 byte-order mark on line 1 is skipped.
- **Quoting.** A hand-written splitter (`split_csv`) supports quoted fields with doubled quotes, rejects a stray quote inside an unquoted field, and rejects text after a closing quote.
- **Limits.** Lines longer than 1024 bytes, fields longer than the internal maximum, embedded NUL bytes and more than eight fields are rejected.
- **Validation.** Timestamps are ISO-8601 UTC; caller and callee are 1 to 31 characters from a restricted set; duration is 0 to 86,400 s; status code 100 to 999; signal strength -150 to 0 dBm; packet loss 0 to 100 %.
- **Strict versus lenient.** With `ParseOptions.strict` non-zero the first invalid row fails the whole parse; otherwise invalid rows are counted and skipped.
- **Parse report.** `ParseReport` records lines read, records parsed, records rejected, whether a header was present, and the line number, status and message of the first error. On failure `records_parsed` is reset to 0.

The default limit is 10,000,000 records (`CA_DEFAULT_MAX_RECORDS`); exceeding it returns `CA_ERR_LIMIT`.

5.4 Status-code classification and rate formulas

> **Warning.** The ranges and formulas below are Aegis-Net's own conventions. They are SIP-inspired but are not a universal telecom standard, and real networks use many other cause-code schemes.

Table 5. Status-code classes (`call_outcome_for_status` in `src/utils.c`)

Range	Class	Meaning in this project
200 to 299	`CALL_COMPLETED`	Completed normally
900 to 999	`CALL_DROPPED`	Established, then ended abnormally (for example 901 radio link failure)
300 to 899	`CALL_SETUP_FAILED`	Never established (for example 408 timeout, 486 busy, 503 unavailable)
100 to 199	`CALL_UNKNOWN`	Provisional; counts as neither success nor failure

$$success_rate = completed / total$$

$$drop_rate = dropped / (completed + dropped)$$

$$setup_failure_rate = setup_failed / total$$

Drops are measured against *established* calls, the conventional definition. Example: 100 calls, of which 90 complete, 6 fail setup, 3 drop and 1 is provisional. Then success_rate = 90/100 = 0.90, drop_rate = 3/(90+3) = 0.0323, setup_failure_rate = 6/100 = 0.06. The code is short enough to read in full:

```
CallOutcome call_outcome_for_status(int status_code)
{
    if (status_code >= 200 && status_code <= 299) {
        return CALL_COMPLETED;
    }
    if (status_code >= 900 && status_code <= 999) {
        return CALL_DROPPED;
    }
    if (status_code >= 300 && status_code <= 899) {
        return CALL_SETUP_FAILED;
    }
    return CALL_UNKNOWN;
}
```

Note the order: the dropped range is tested before the wide 300 to 899 range only for readability; the ranges do not overlap.

5.5 Aggregations

Statistics are computed with running accumulators (`CaAccumulator` in `ca_internal.h`) that keep integer sums with overflow checks (`ca_accumulator_add` returns `CA_ERR_OVERFLOW` rather than wrapping) and a Welford running mean and second moment for duration spread. Cell statistics sort pointers to records by cell id; time windows align to multiples of the window size; caller statistics count distinct callees; failure-code statistics are sorted by count descending then code ascending. Results are sorted deterministically so tests and reports are stable.

5.6 Call anomaly detection

`src/anomaly.c` compares each group with its peers using a *robust z-score*:

$$z = (x - median) / max(1.4826 \times MAD, floor)$$

where MAD is the median absolute deviation, the constant 1.4826 makes MAD comparable to a standard deviation for normally distributed data, and `floor` prevents tiny, tightly clustered baselines from inflating z. The score is

$$score = 1 - 2^{(-z / threshold)}$$

so a group exactly at the threshold scores 0.5 (`ca_anomaly_score`). Severity bands are score at least 0.9 critical, at least 0.75 high, at least 0.6 medium, otherwise low (`ca_severity_for_score`).

> **Note.** External background (consulted): the NIST/SEMATECH e-Handbook defines a modified z-score as 0.6745 times the deviation from the median divided by MAD, and suggests flagging absolute values above 3.5 as potential outliers [R4]. 0.6745 is the reciprocal of 1.4826, and the library's default group threshold is also 3.5. Aegis-Net adds floors and minimum absolute deltas on top; it does not claim to implement the handbook procedure unchanged.

The defaults (`anomaly_config_default`): group z threshold 3.5, per-call duration threshold 10.0, at least 3 peer groups, at least 20 calls per cell, 10 per window, 20 per caller, 30 established calls for duration outliers, 300-second windows, minimum absolute deltas of 5 percentage points for rates, 6 dBm for signal, 3 points for packet loss, volume ratio 2.0, short-call ratio 0.6, and at most 1000 anomalies kept. A group is flagged only when *both* z reaches the threshold *and* the absolute deviation reaches the minimum delta. Anomaly types: cell drop rate, cell setup-failure rate, cell low signal, cell packet loss, window drop rate, window volume spike, caller high volume, and call duration.

The cell detector shows the pattern:

```
for (size_t k = 0; k < n; k++) {
    const CellStatistics *c = &cells[idx[k]];
    CallAnomaly a;
    if (z[k] < cfg->z_threshold ||
        (double)directions[m] * (values[k] - median) < deltas[m]) {
        continue;
    }
    init_anomaly(&a, types[m], c->cell_id, values[k], median, z[k], cfg->z_threshold,
                 cell_sample(c, (CellMetric)m));
    ...
}
```

`directions[m]` is +1 when a higher value is worse (drop rate) and -1 when lower is worse (signal), so one comparison handles both. In the seeded run the library flagged setup-failure and packet-loss anomalies on exactly CELL-101 to CELL-104, for example "Cell CELL-103 setup failure rate 14.2% vs peer median 0.8% (1419 calls)" (Figure Figure 4).

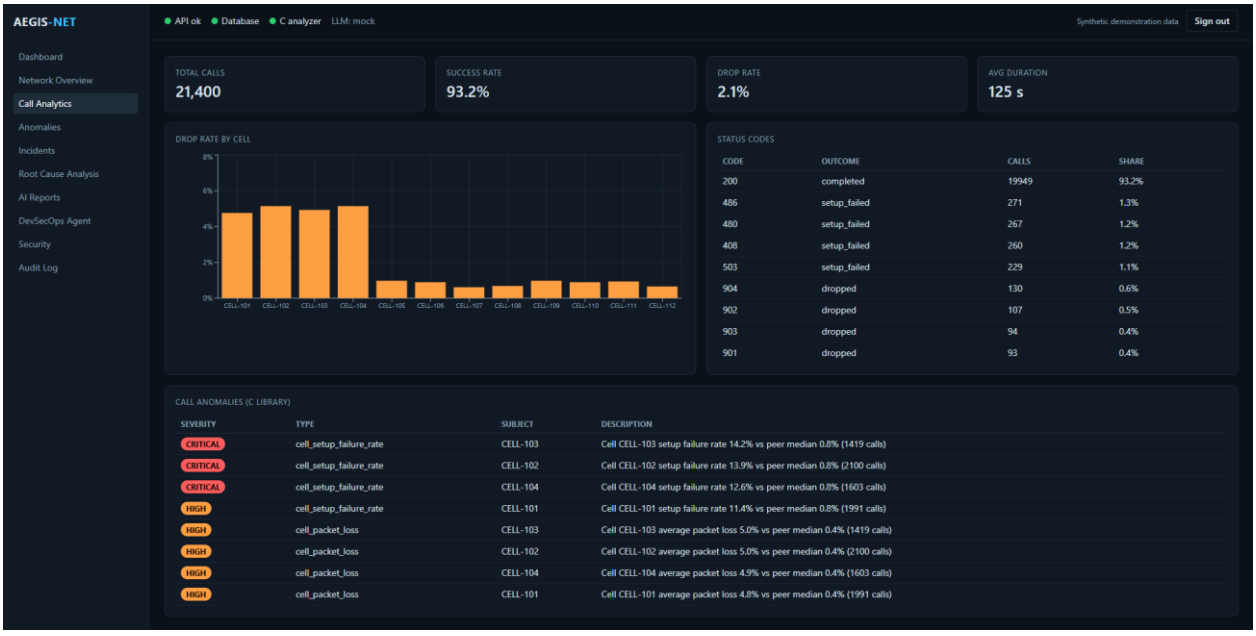


Figure 4. Call Analytics page: totals, drop rate by cell, status codes and C-library anomalies

In Figure 4 the four tall bars are CELL-101 to CELL-104 (about 5 % drop rate against under 1 % for the rest); the status-code table shows the project's classes (200 completed; 486, 480, 408, 503 setup failures; 901 to 904 drops); and the anomaly table at the bottom is the C library's output labelled "(C LIBRARY)". Source: supplied screenshot document, synthetic data.

5.7 Memory allocation and ownership

All allocation goes through wrappers (`ca_malloc`, `ca_calloc`, `ca_realloc_array`, `ca_free`). `ca_realloc_array` fails on multiplication overflow, which blocks the classic "count times size wraps around" bug. Under the build flag `CA_FAULT_INJECTION` the wrappers can be told to fail the *n*th allocation, which lets the test `test_oom_all_operations` force every allocation site to fail in turn and check that each returns `CA_ERR_NOMEM`, frees what it had and leaves outputs NULL. Multi-step functions use a single cleanup label (`goto done`) so every path releases its temporary buffers, as the cell detector above does.

Native memory ownership matters because a leak or double free in a long-running service is hard to see from Python, and because the Python process cannot be protected by garbage collection from native mistakes.

5.8 Build and test layers

Table 6. C test and build layers

Layer	What it checks	What it does not prove
Strict warnings, `-Werror`	Implicit conversions, shadowing, missing prototypes, unused values	Logic errors
Unit tests (28 functions)	Parsing, malformed rows, statistics, anomalies, NULL arguments	Behaviour on inputs nobody thought of
ASan, UBSan, LSan build	Out-of-bounds access, undefined behaviour, leaks on tested paths	Untested paths; other compilers
Fault-injection build	Every allocation site fails cleanly	Failures of other system calls
Hand-run mutation checks	The plan records mutations of gates in a scratch copy; they were detected	There is no mutation tool in the repository, so this is not repeatable by command

6 Python and C Integration

> **Implementation status.** VERIFIED `backend/tests/test_binding_abi_and_ownership.py` (28 tests) and `test_call_analyzer_binding.py` (61 tests) passed in the full run: 352 passed, 3 skipped.

6.1 ctypes from first principles

A **shared library** (`libcallanalyzer.so` on Linux) is compiled code that other programs load at run time. Python's `ctypes` module can load one and call its functions, but C functions carry no type information at run time, so Python must be told each function's signature. The official documentation describes `argtypes` (the C types a function expects, so ctypes can check and convert arguments) and `restype` (the return type, default `int`), and structures declared as a subclass of `Structure` with a `_fields_` list, laid out as the C compiler would [R1].

If a signature is wrong, nothing warns you: a 64-bit `size_t` passed as a 32-bit `int` silently truncates; a pointer returned as `int` is cut to 32 bits and crashes later. This is why Aegis-Net's binding is paranoid.

6.2 Structure layout and the ABI

The **application binary interface (ABI)** is the machine-level contract: field order, sizes, alignment and padding. In `backend/call_analyzer/_native.py` each C struct has a Python mirror. For example:

```
class CallRecord(ctypes.Structure):
    _fields_ = [
        ("timestamp", ctypes.c_char * CA_FIELD_LEN),
        ("caller", ctypes.c_char * CA_FIELD_LEN),
        ("callee", ctypes.c_char * CA_FIELD_LEN),
        ("cell_id", ctypes.c_char * CA_FIELD_LEN),
        ("duration", c_int),
        ("status_code", c_int),
        ("signal_strength", c_int),
        ("packet_loss", c_int),
        ("epoch_seconds", c_int64),
    ]
```

Four 32-byte arrays occupy 128 bytes; four 4-byte ints bring the offset to 144, already a multiple of 8, so the 8-byte `epoch_seconds` needs no padding and the struct is 152 bytes. The binding does not assume this arithmetic. At load it calls `call_analyzer_struct_sizes` and compares every entry with `ctypes.sizeof` of the mirror; any mismatch raises `LibraryLoadError`. It also requires major version 1. The tests go further: they compile a C probe using `offsetof`, `sizeof` and `_Generic` against the header and compare each field's name, order, offset, size and type class with the Python mirror.

6.3 Function prototypes and pointers

`_PROTOTYPES` lists `(restype, argtypes)` for every function used, for example `calculate_cell_statistics` takes a `POINTER(CallRecord)`, a `size_t`, a pointer to a `POINTER(CellStatistics)` and a pointer to a `size_t`. Out-parameters are created as `ctypes.POINTER(T)()` (a NULL pointer) and passed with `ctypes.byref`.

6.4 Native allocation, freeing and exception mapping

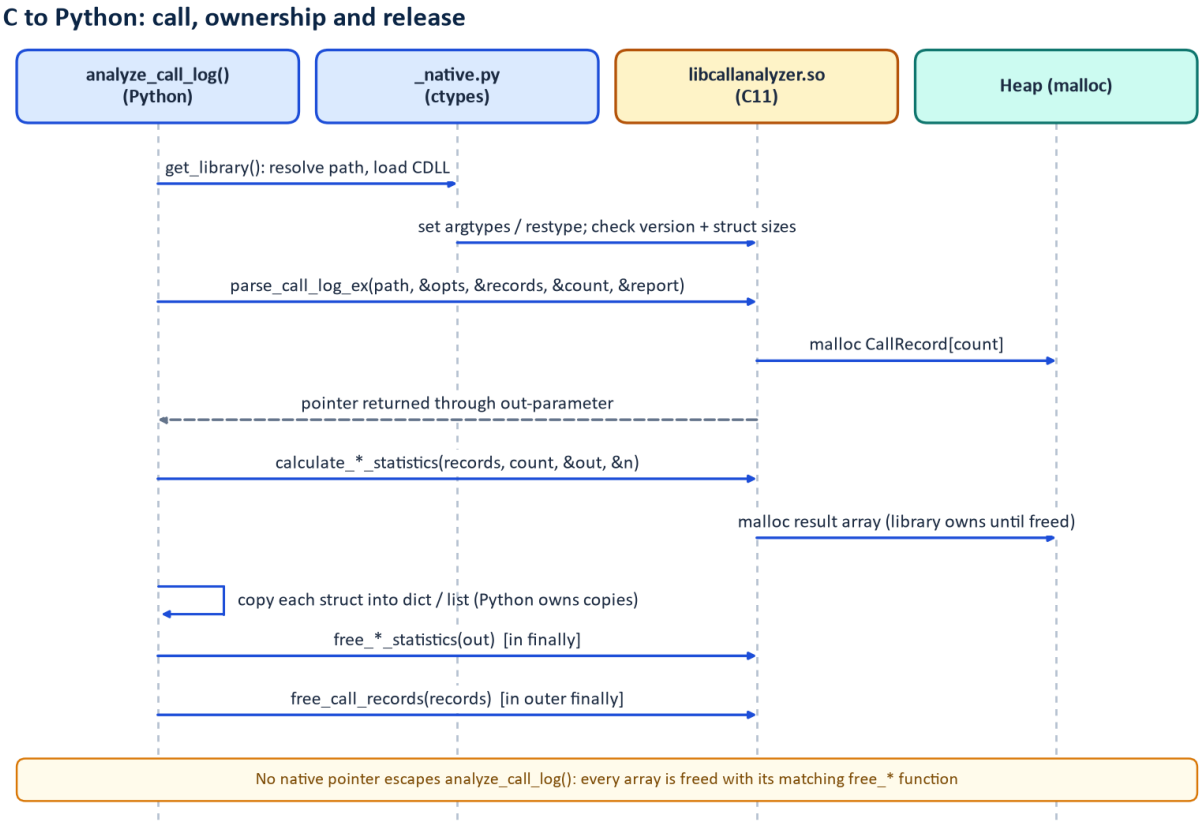


Figure 5. C to Python call, ownership and release

Figure 5 shows the rule: the library allocates, Python copies results into dictionaries, and then frees with the matching `free_*`. The helper `_collect_array` does this in a `try/finally`, so the array is freed even if conversion raises, relying on the C guarantee that `out` is NULL on failure and `free(NULL)` is harmless. The outer `finally` in `analyze_call_log` calls `free_call_records` after all analyses. Result: no native pointer escapes.

Non-zero statuses become exceptions through `error_from_status`: a `CallAnalyzerError` base class carrying `code`, `code_name`, the library's own message and, for parse failures, the parse report, with subclasses such as `CallLogParseError`, `OutOfMemoryError`, `CallAnalyzerOverflowError` and `LimitExceededError`, plus `LibraryLoadError` for loading failures.

6.5 Library discovery

`resolve_library_path` tries, in order: the `CALL_ANALYZER_LIB` environment variable (a non-blank value must point to a file or loading fails; there is no silent fallback), `c-library/build/libcallanalyzer.so` (from `make c-build`), then `/usr/local/lib/libcallanalyzer.so` (the Docker image). It always loads by absolute path, never through the dynamic linker search path.

6.6 Why integer width and overflow validation matter

Before calling C, the binding rejects arguments ctypes would silently wrap: `window_seconds` outside the int64 range, a negative `max_records`, overrides that overflow a C `double` or `size_t`, NUL bytes in the path, and unknown `anomaly_config` keys. Python integers are unbounded; C integers are not. Without these checks, `window_seconds = 2**63` would become a negative number inside C.

6.7 Why Python orchestrates and C computes

The binding module states the rule: *all computation happens in C*. Python converts structs, names enum values using the C `*_name` functions, and takes anomaly defaults from `anomaly_config_default`. The only Python-side derivation is `suspicious_time_windows`, which groups the window-category anomalies C returned and joins them to that window's C statistics, and it raises an error rather than invent a value if C returned a window with no statistics. Keeping computation in one place prevents two implementations from drifting apart.

7 Synthetic Network Telemetry

> **Implementation status.** SIMULATED The simulator is a testing instrument. VERIFIED Its determinism and causal properties are tested (`test_simulation.py`, 31 tests).

7.1 What it produces

`simulate(Scenario)` returns, for 12 cells at one-minute resolution (default 360 minutes starting 2026-03-01T12:00Z): the 17 KPIs of Chapter 2, operational events (deployments, configuration changes, security events, application logs), deployment records and a ground-truth description used only by tests and the demo. Detection and RCA never read the ground truth.

7.2 Baselines, load cycle, noise and determinism

- **Diurnal load.** `diurnal_factor` scales users by `1 + 0.25 sin(2 pi (hour - 12) / 24)`, so load peaks near 18:00 UTC and bottoms near 06:00.
- **Per-cell profile.** Each cell draws base users (160 to 260), base RSRP (-94 to -88 dBm) and base SINR (15 to 20 dB).
- **Noise.** Every metric carries autoregressive AR(1) noise with coefficient 0.7, so consecutive minutes are correlated rather than independent draws, as in real series.
- **Seeding.** NumPy `SeedSequence` spawns independent streams for profiles, incidents, background events and each cell, so identical scenarios give identical output. `SimulationResult.fingerprint()` is a SHA-256 over all values and events; the seeded scenario's fingerprint begins `aed745627289`.

7.3 The causal model

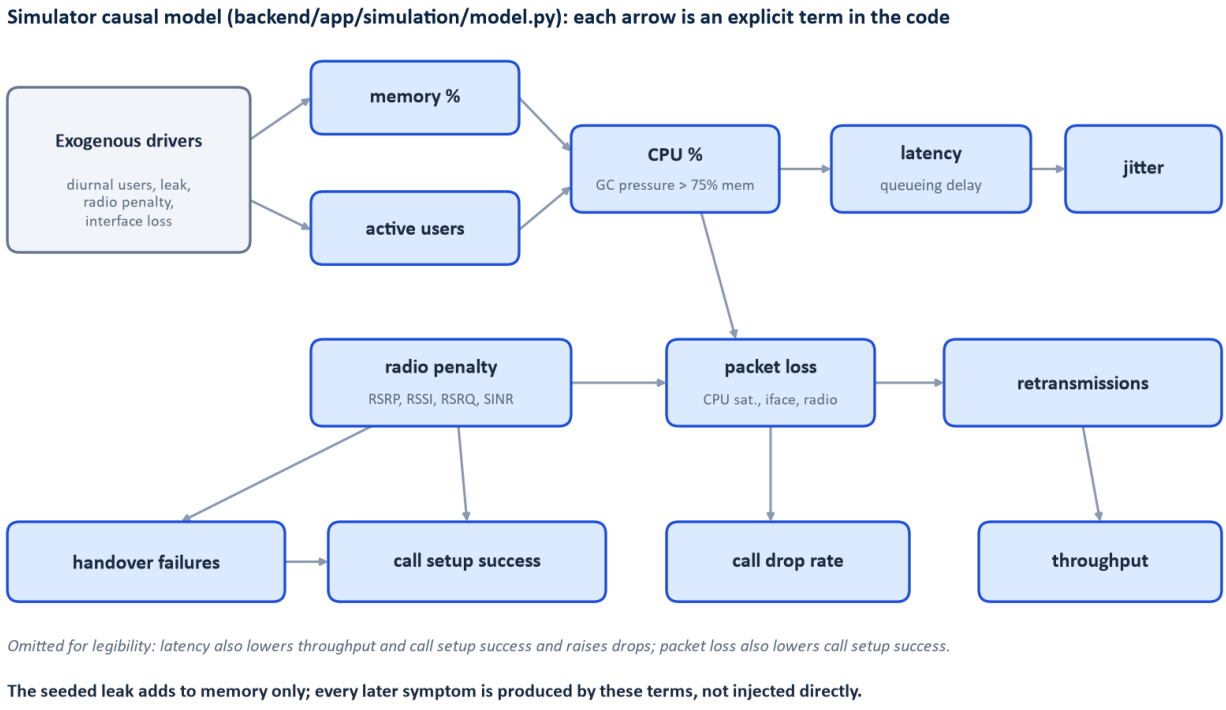


Figure 6. The simulator's causal chain

The terms in `model.py` implement Figure 6. In brief: memory is `50 + 8 × load` plus any leak; CPU is `22 + 35 × load` plus garbage-collection pressure `1.8 × max(0, memory - 75)`; queueing delay is `4 u / (1 - u)` ms for

utilisation `u = CPU/100`, capped at 400 ms, added to an 18 ms base for latency; packet loss adds a logistic overflow term that switches on near 93 % CPU. Because incident injectors change only drivers, the downstream symptoms emerge from these formulas.

7.4 Incident scenarios

Table 7. The nine incident types (`incidents.py`)

Type	Scope	Driver changed
overloaded_base_station	cell	users x (1 + 1.3 ramp)
failing_network_interface	cell	random link flaps add interface loss
packet_loss_burst	cell	+11 percentage points loss
abnormal_traffic_spike	cell	users x 1.8
degraded_radio_signal	cell	up to 14 dB radio penalty
software_deployment_regression	node	+38 CPU points; creates a deployment record
memory_leak	node	memory leak about 0.8 points per minute, capped at 41; creates a deployment record
security_attack	cell	users x 1.9, +22 CPU, 3 to 8 IDS events per minute
misconfiguration	cell	handover and setup penalties; creates a config-change event

7.5 The seeded memory-leak and deployment incident

`seeded_memory_leak_scenario(seed=7)` deploys packet-gateway 2.4.1 to `pgw-1` at minute 125, which is 14:05, with a memory leak lasting 235 minutes. The leak starts after a one-minute warm-up. Warning logs ("GC pause ... ms in packet-processing worker") begin once the leak passes 30 points. A benign "hotfix1" deployment to `pgw-3` at 13:10 is a deliberate distractor.

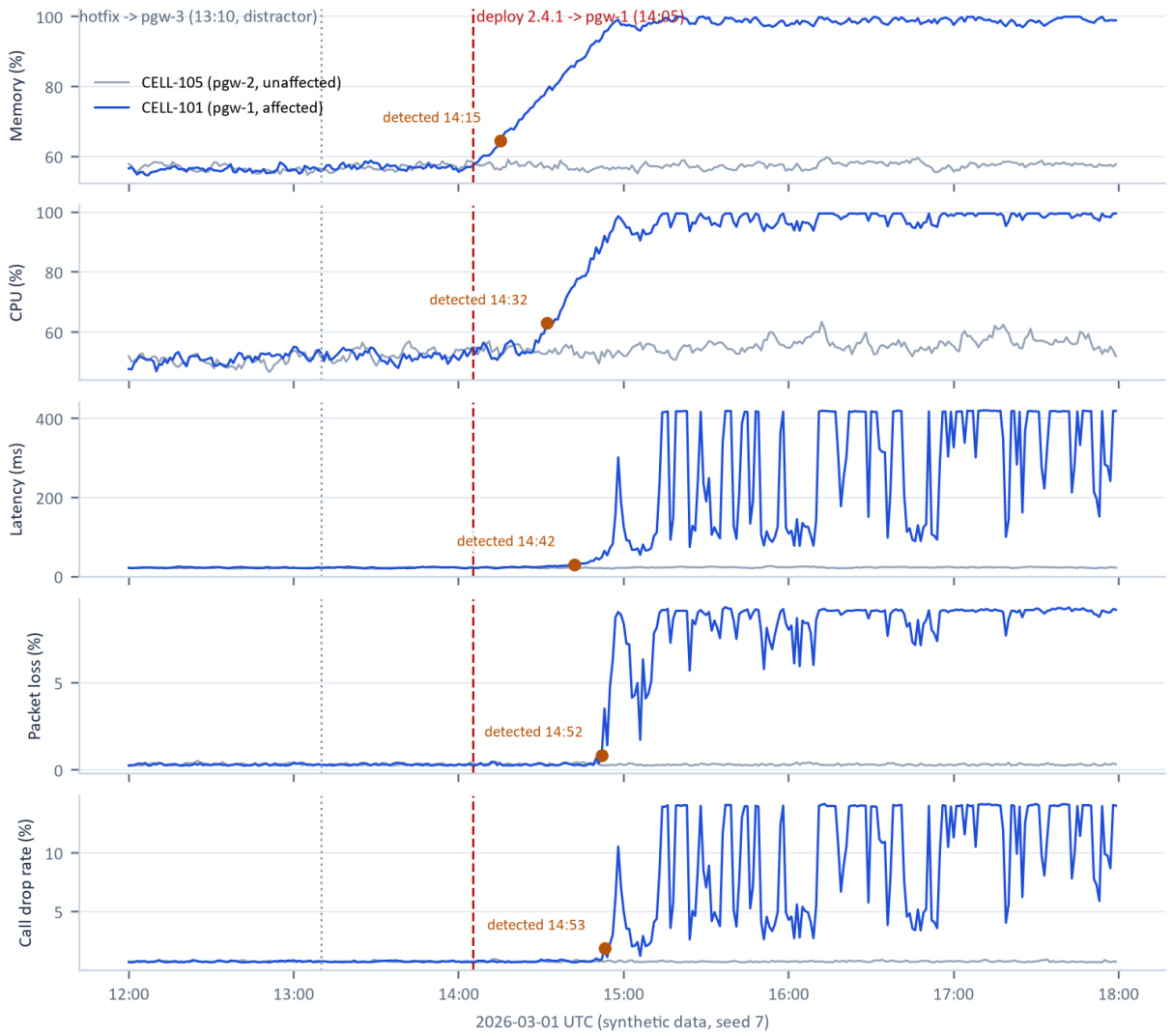


Figure 7. Seeded run, affected cell versus an unaffected control cell

Figure 7 is drawn from data computed by the repository's simulator and detectors (script `scripts/documentation/extract_evidence.py`). On CELL-101 memory climbs from the 14:05 deployment, CPU follows once memory passes 75 %, then latency, packet loss and call drops. CELL-105 on pgw-2 is flat. The amber markers are where the detectors first flagged CELL-101.

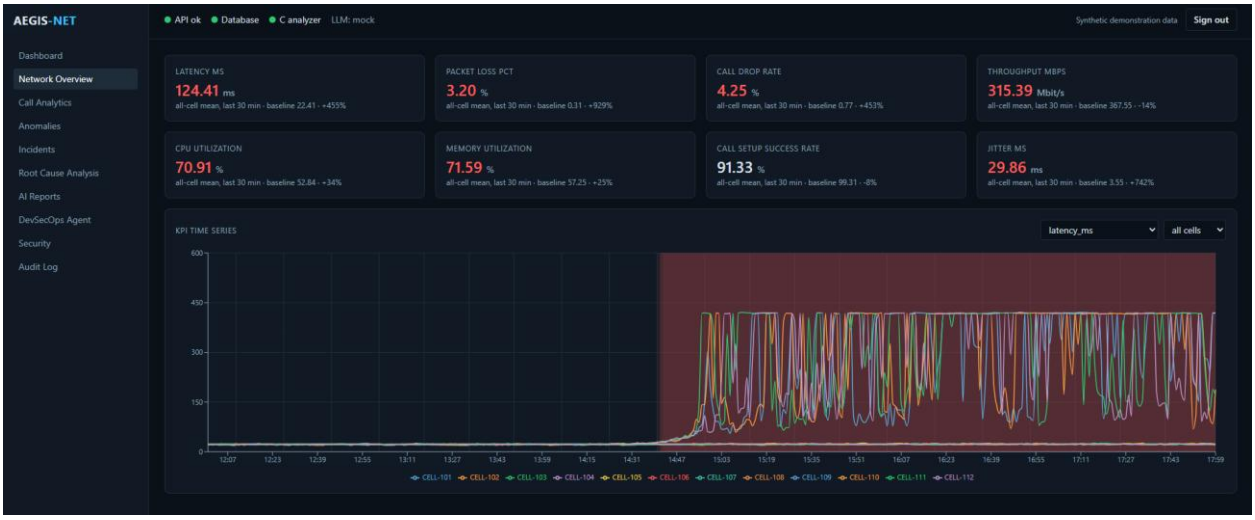


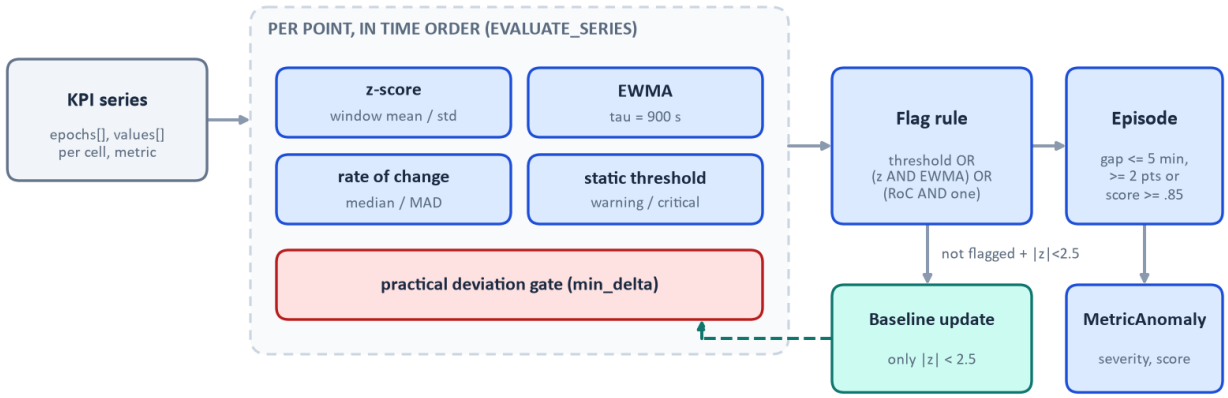
Figure 8. Network Overview page for the same incident

In Figure 8 the eight KPI cards compare the last 30 minutes (all-cell mean) with the baseline: latency 124.41 ms against 22.41 (+455 %), packet loss 3.20 % against 0.31, call drop rate 4.25 % against 0.77, memory 71.59 % against 57.25. The red shaded region is the detected anomaly window. All twelve cells are plotted; four climb, eight stay flat. The all-cell means are lower than the affected-cell means because eight healthy cells dilute them.

> **Warning.** The simulator is a testing instrument. Detector performance measured on it does not show that the same performance would hold on real production telemetry. Thresholds were tuned on this simulator and on generic synthetic series.

8 Anomaly Detection, From Basic to Advanced

> **Implementation status.** VERIFIED Implemented in `backend/app/analytics/detectors.py`, `engine.py` and `config.py`; 24 tests in `test_analytics.py`. Measured false-positive rate on healthy data recorded in the plan: 0.27 episodes per 100 series-days, none high or critical (a recorded result, not re-run here).



Scoring: score = 1 - 2^(-z / threshold) (0.5 at the threshold)

Severity from score: >= 0.9 critical, >= 0.75 high, >= 0.6 medium, otherwise low.

Re-baseline: after 6 h with no admitted points, the baseline is re-seeded from current values.

Baseline freeze: flagged or extreme points never update the baseline, so a slow ramp cannot hide itself.

Source: backend/app/analytics/detectors.py, engine.py, config.py.

Figure 9. Detector processing pipeline

Figure 9 summarises `evaluate_series`, which walks each series in time order and, for every point, computes four detector scores, decides whether the point is flagged, and (only if it is clearly normal) lets the point update the baseline.

8.1 Worked example used throughout

Take ten one-minute readings of a KPI: 50, 51, 49, 50, 52, 50, 51, 49, 50, then 80.

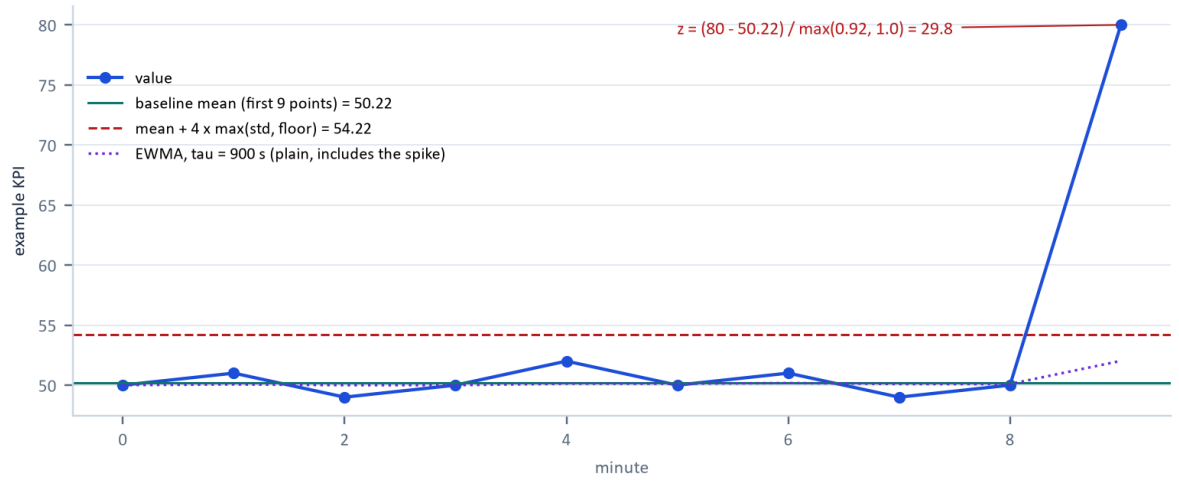


Figure 10. The worked example, with baseline, threshold and EWMA

8.2 Moving average and rolling standard deviation

Intuition. What has this series typically looked like recently? **Definition.** Over a trailing time window W (60 minutes by default), the mean is the sum of values divided by their count, and the standard deviation is the square root of the average squared distance from the mean. **Example.** For the first nine readings the mean is 50.22 and the standard deviation 0.92. **Parameters.** `window_seconds = 3600`, `min_baseline_points = 20` before z-detection starts. **Implementation.** `_Baseline` keeps a deque of (time, value) pairs plus running sums, evicting old points by *time* rather than count so gaps and irregular sampling work. The helpers `moving_average` and `rolling_std` exist for charts and reports. **Strengths.** Simple, explainable. **Weaknesses.** A long window reacts slowly; a short one learns the anomaly. **Tests.** Detector unit tests on synthetic steps, ramps and gaps.

8.3 Z-score

Intuition. How many "typical wobbles" away is this reading? **Definition.**

$$z = (value - mean) / max(std, scale_floor)$$

oriented so that positive means the *bad* direction for that KPI. The floor is `max(noise_floor, relative_floor x |mean|)` per metric (`config.DEFAULT_RULES`), so a very quiet series cannot turn trivial changes into huge z. **Example.** Using latency's noise floor of 1.0: z = (80 - 50.22) / max(0.92, 1.0) = 29.8. **How it is used.** The z-score detector's score is `1 - 2^(-z/4)` (threshold 4.0), but only counts if the deviation also exceeds the metric's practical `min_delta` (6 ms for latency). **Strengths.** Standard and interpretable. **Weaknesses.** Assumes roughly bell-shaped noise; one big outlier inflates the standard deviation; ramps can drag the mean along, which is why the baseline admits only clearly normal points. **False positives.** Seasonal peaks the window has not seen. **Tests.** Step, ramp, spike and noise cases.

8.4 EWMA

Intuition. A running average that forgets old data smoothly. **Definition.** `EWMA_t = alpha x Y_t + (1 - alpha) x EWMA_(t-1)`, the form given in the NIST handbook with smoothing constant between 0 and 1 [R2]. Aegis-Net makes alpha *time-aware*: `alpha = 1 - exp(-dt / tau)` with tau = 900 s, so a one-minute step gives alpha of about 0.0645. A matching exponentially weighted variance gives a standard deviation. **Example.** Starting at 50 with tiny wobbles the EWMA stays near 50.1; if the 80 were admitted it would jump only to 52.0 (the plain `ewma` helper output for this series, last point), showing how slowly an EWMA moves. **How it is used.** Its z is computed against the EWMA mean and variance; the detector flags only when z-score *and* EWMA agree (z at least 4 on both). **Strengths.** Cheap, adapts to slow drift. **Weaknesses.** Lags a sudden shift; the detector therefore never feeds flagged points into it. **Tests.** Time-awareness with irregular spacing.

8.5 Absolute thresholds

Intuition. Some values are bad no matter what is normal. **Definition.** Per-KPI warning and critical levels, for example latency 100 ms and 200 ms, packet loss 2 % and 5 %, CPU 85 % and 95 %, memory 85 % and 95 %. A breach scores 0.7 (warning) or 0.95 (critical) and flags the point on its own. **Strengths.** Catches a series that has *always* been bad, and shows operators familiar numbers. **Weaknesses.** Fixed numbers encode judgement made for the simulator. **False positives.** A genuine, intended load increase.

8.6 Rate of change

Intuition. Is the series moving much faster than it usually moves? **Definition.** Per-minute change compared with the median and MAD of recent healthy changes: `z = (rate - median) / max(1.4826 × MAD, floor)`, threshold 5.0, requiring at least 20 healthy changes and the practical-deviation gate. **Strengths.** Catches onset quickly. **Weaknesses.** Noisy at high sampling rates; on its own it does not flag, it needs support from z-score or EWMA.

8.7 Scoring, severity, episodes and baselines

Figure 11 plots the score against z for the C library (threshold 3.5), the Python z-score and EWMA detectors (4.0) and rate of change (5.0). Worked values at threshold 4: z = 4 gives 0.5, z = 8 gives 0.75, z = 12 gives 0.875.

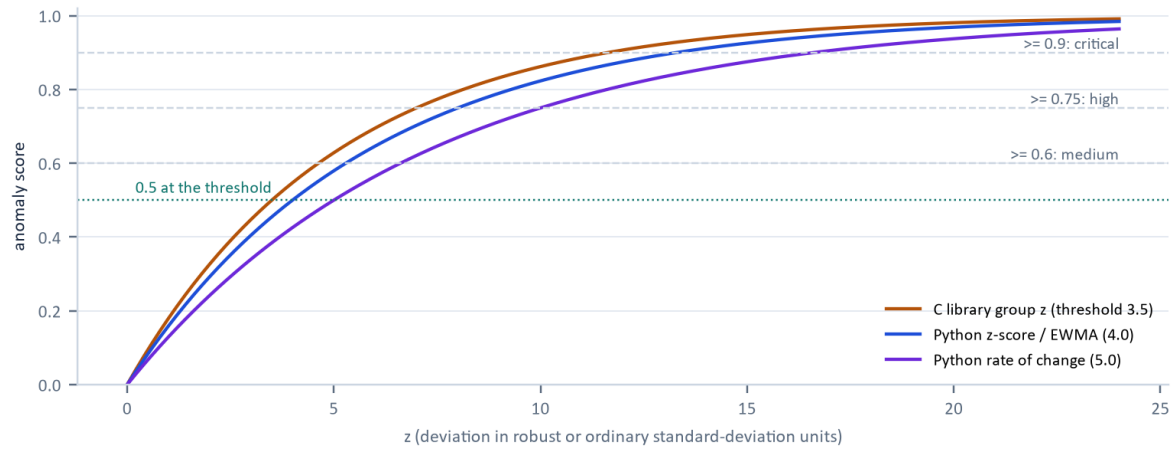


Figure 11. Score as a function of z, with severity bands

- **Flag rule.** A point is flagged if a static threshold is breached, or z-score and EWMA both reach 0.5, or rate of change reaches 0.5 together with one of them.
- **Severity.** `severity_for_score`: score at least 0.9 critical, 0.75 high, 0.6 medium, else low.
- **Episodes.** Flagged points closer than 300 seconds join one episode; an episode needs at least two points, or a single point scoring at least 0.85. The anomaly carries onset time, the most extreme value, the baseline at onset, the highest score, the detectors that fired and `related_metrics` (same-cell episodes overlapping within 15 minutes).
- **Baseline freeze and re-baselining.** Only points with |z| below 2.5 update the baseline, so neither a slow ramp nor a good-direction excursion can contaminate it. After six hours without any admitted point the series is re-seeded from current values, so a permanent level shift is eventually accepted as the new normal. Until then a permanent shift keeps being reported.

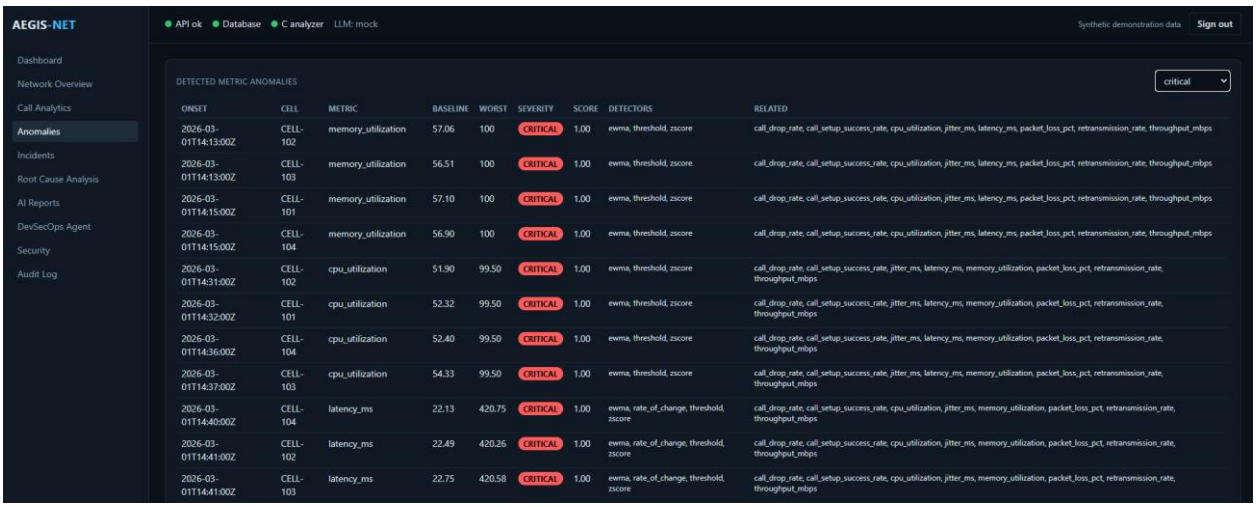


Figure 12. Anomalies page filtered to critical severity

Figure 12 lists, per cell, the onset time, metric, baseline, worst value, severity, score and which detectors fired. Reading the first rows: memory on CELL-102 and CELL-103 starts at 14:13 and on CELL-101 and CELL-104 at 14:15, baseline about 57, worst value 100, detectors ewma, threshold and zscore. The scores are all 1.00 because the extreme point saturates the score. The "related" column lists the other metrics that moved on the same cell.

8.8 Detectors versus the simulator

The simulator *creates* anomalies by changing drivers; the detectors know nothing about drivers. `analytics` modules import only the KPI catalogue from the simulator, and a test enforces that. This separation is what makes it meaningful that the detectors found the leak.

8.9 Strengths, failure modes and limits across all detectors

- All four are interpretable: each flag can be traced to numbers.
- Thresholds and floors are hand-set per metric; there is no learning.
- Slow drifts that stay under thresholds for hours can be absorbed by the baseline.
- Multivariate patterns (two KPIs jointly unusual) are not modelled; correlation handles that at the incident level.

> **Note.** Future work (not implemented): Isolation Forest, an ensemble of random trees that isolates unusual points quickly, and DBSCAN, a density-based clustering method that labels sparse points as noise, are optional in the specification and are not part of Aegis-Net NOT IMPLEMENTED. They are mentioned only as candidates for Chapter 20.

9 Correlation and Root-Cause Reasoning

> **Implementation status.** VERIFIED `backend/app/analytics/correlation.py` and `causal_graph.py`; 29 tests in `test_correlation.py`. The recorded healthy-data check found 0 incidents over 20 healthy days (a recorded result, not re-run here). The seeded outputs below were recomputed for this handbook.

9.1 Why symptoms share a cause

Nine KPIs turned critical in the seeded run, but there was one fault. A deployment leaked memory; memory pressure forced garbage collection and raised CPU; CPU raised queueing delay (latency, jitter); congestion lost packets; lost packets caused retransmissions, dropped calls and lower throughput. An operator who sees nine alerts needs the engine to group them and say which came first and what changed just before.

9.2 What the engine actually computes

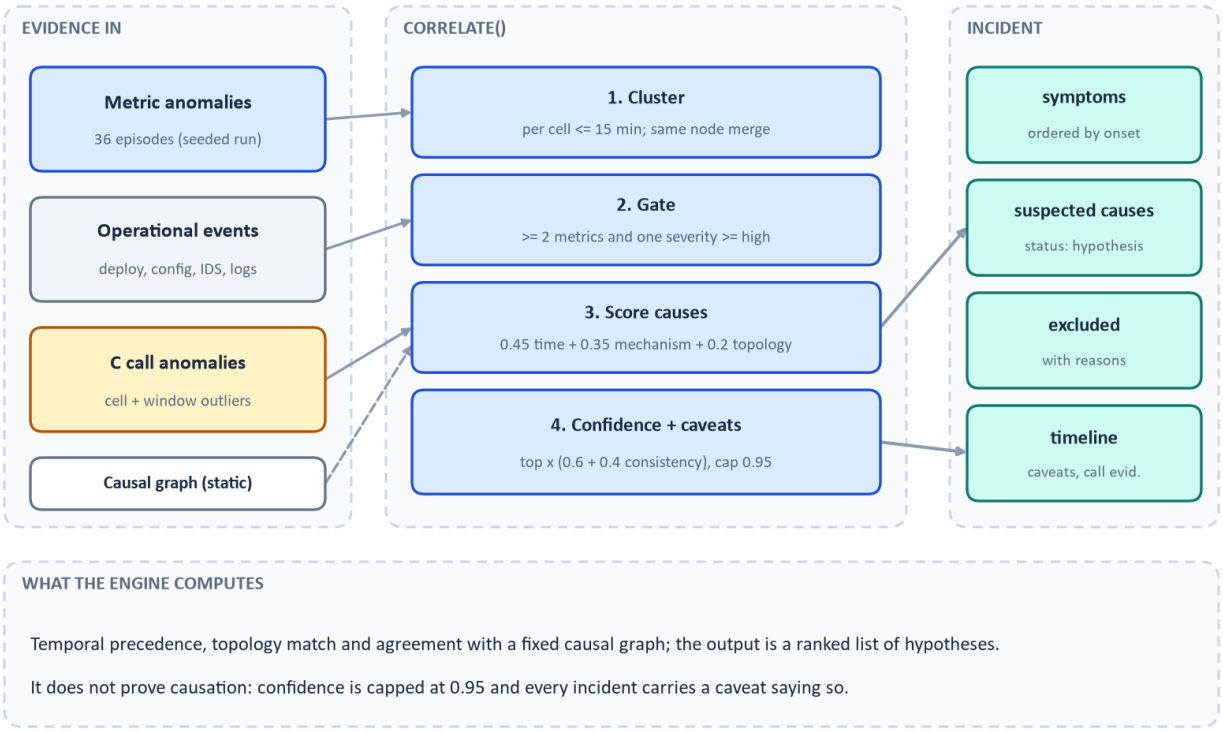


Figure 13. Evidence in, ranked hypotheses out

`correlate()` works in five steps (Figure 13).

1. **Clustering.** Anomaly episodes on one cell whose start lies within 15 minutes of the cluster's span form a cell cluster. Cell clusters on the *same* node merge if their onsets are within 15 minutes; clusters on different nodes never merge.
2. **Incident gate.** A cluster becomes an incident only with at least two distinct symptom metrics and at least one symptom of severity high or above. This is why coincidental noise does not open incidents.
3. **Candidate causes.** Each candidate gets

$$score = 0.45 \times temporal + 0.35 \times mechanism + 0.20 \times topology$$

where *temporal* is `exp(-lead / 30 min)` for an event up to two hours before onset (0.3 for an event within five minutes after onset, 0.5 for symptom-only causes); *mechanism* is half the share of the earliest symptoms (within two

minutes of onset) that the cause is expected to produce first plus half the share of all symptoms the cause explains in the causal graph; and *topology* is 1.0 when the event targets an affected node or cell and 0.5 otherwise. Event-backed classes are deployment regression, configuration change and security attack (needs at least 10 IDS events); symptom-only classes are memory exhaustion, traffic overload, radio degradation and transport fault.

1. **Exclusion.** Deployments and changes on other elements are listed in `excluded_candidates` with a reason.
2. **Confidence** `= top score × (0.6 + 0.4 × chain consistency)`, multiplied by 0.9 if the top two scores are within 0.1, capped at 0.95. *Chain consistency* is the share of effects that are preceded (within a one-minute tolerance) by at least one observed upstream cause in the fixed causal graph (`causal_graph.EDGES`).

Warning. Time ordering plus a hand-written graph is evidence, not proof. Statistical correlation does not establish causation, and the engine says so: every suspected cause has `status: "hypothesis"`, the incident carries a caveat to that effect, and confidence is capped.

The engine also records, for each adjacent pair of symptoms, the onset lag and (when series are available) the best lag of the Pearson correlation of first differences. In the seeded run that lag came out as 0 for most pairs (cross-correlation values from 0.44 to 1.0), so the *onset ordering*, not the cross-correlation lag, carries the temporal story. The report labels the sequence as temporal, not causal.

9.3 The seeded evidence chain

Table 8. Verified symptom onsets for INC-2026-001 (first onset across the four cells; recomputed for this handbook)

Onset (UTC)	Symptom	Baseline	Worst	Lag from previous
14:13	memory_utilization	56.5 %	100 %	(first)
14:31	cpu_utilization	51.9 %	99.5 %	+18 min
14:40	latency_ms	22.1 ms	420.8 ms	+9 min
14:45	jitter_ms	3.7 ms	105.9 ms	+5 min
14:50	packet_loss_pct	0.31 %	9.46 %	+5 min
14:52	retransmission_rate	1.1 %	15.8 %	+2 min
14:53	call_drop_rate	0.80 %	14.2 %	+1 min
14:53	call_setup_success_rate	99.3 %	67.6 %	+0
14:53	throughput_mbps	444 Mbit/s	103.7 Mbit/s	+0

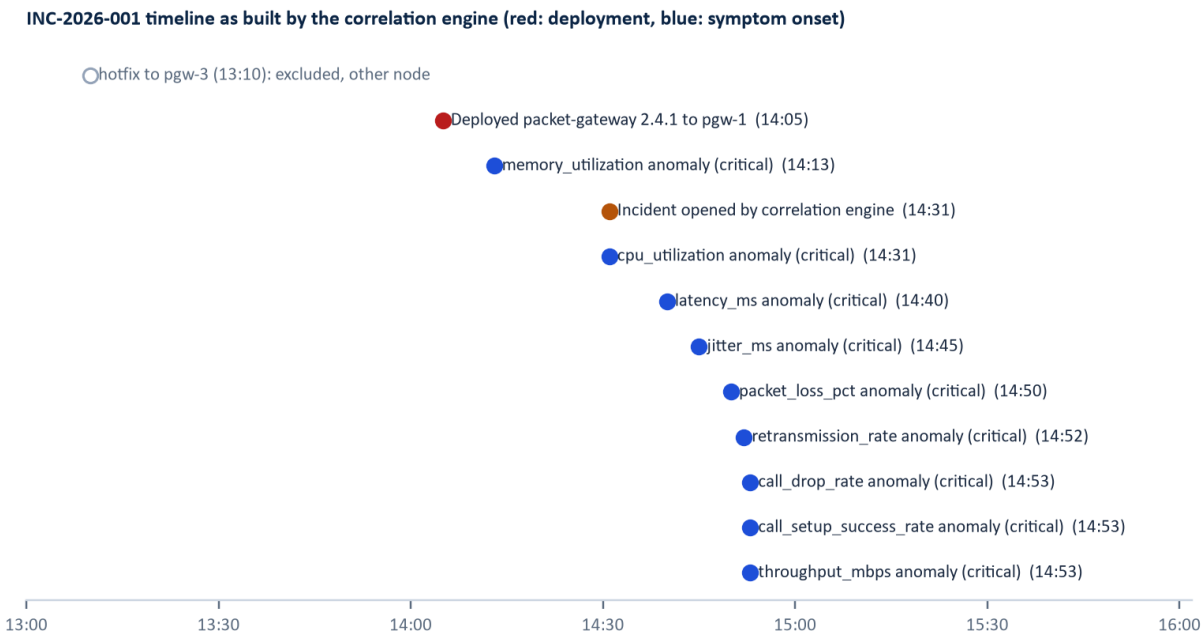


Figure 14. The timeline the engine builds

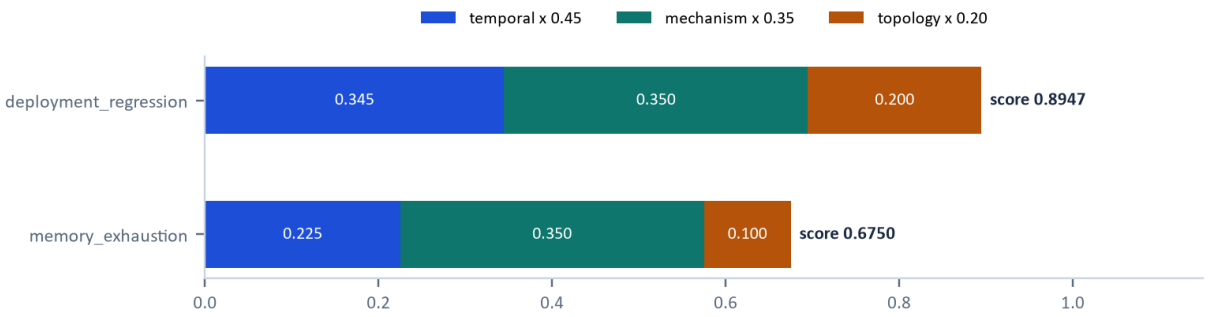


Figure 15. Why deployment_regression outranks memory_exhaustion

Figure 15 decomposes the scores: the deployment 8 minutes earlier on the same node has temporal 0.766 (`exp(-8/30)`, since the deployment preceded the 14:13 onset by 8 minutes), mechanism 1.0, topology 1.0, so `0.45 × 0.7659 + 0.35 × 1 + 0.2 × 1 = 0.8947`. Memory exhaustion has the same mechanism score but only the neutral temporal prior 0.5 and topology 0.5: `0.225 + 0.35 + 0.1 = 0.675`. Chain consistency was 1.0, the margin 0.22 (not a close call), so confidence is 0.8947. The hotfix deployed to `pgw-3` at 13:10 appears under *excluded candidates*: "targets pgw-3, outside the affected nodes ['pgw-1']". That is how the distractor is handled.

> **Note.** The incident timeline shown on the dashboard includes the first ten warning and error application logs on the affected topology (for example "GC pause 382 ms in packet-processing worker" from 14:47), which Figure 14 omits for legibility.

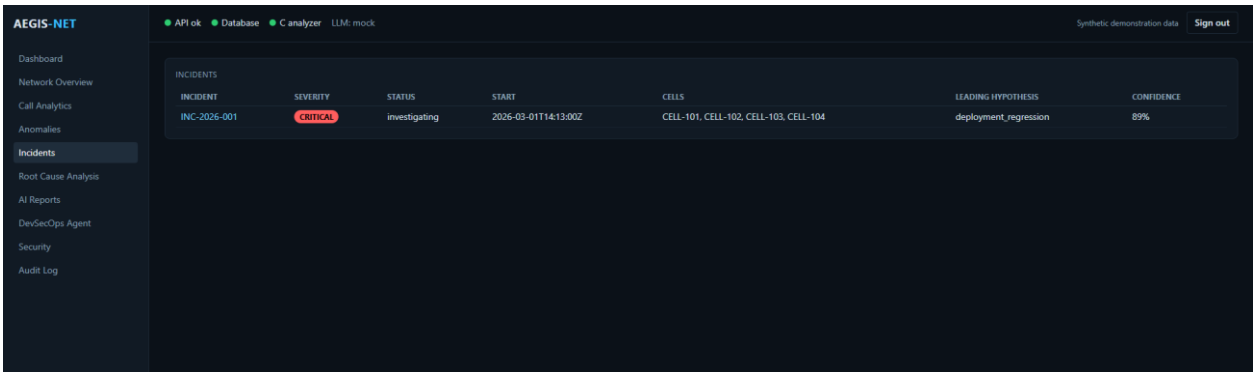


Figure 16. Incidents list: one critical incident with its leading hypothesis

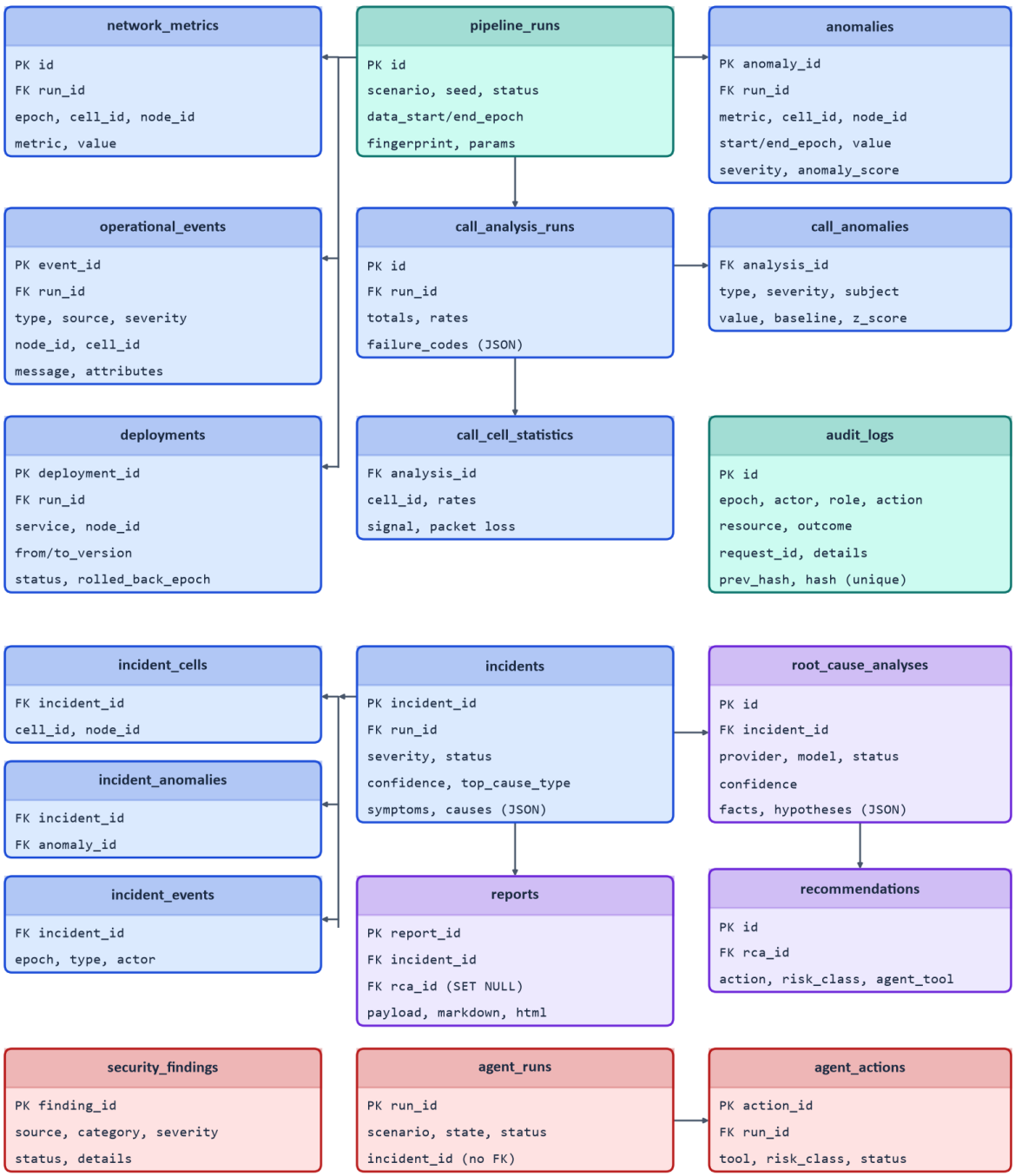
Figure 16 shows the result as an operator sees it: INC-2026-001, critical, status investigating, start 14:13, four cells, leading hypothesis `deployment_regression` at 89 %. The 89 % is the correlation engine's confidence; the RCA page (Chapter 13) shows the RCA engine's own 85 %.

10 Database and Backend API

> **Implementation status.** VERIFIED Models in `backend/app/models/`, migration `backend/alembic/versions/0001_initial_schema.py`, routes in `backend/app/api/`. Tests: `test_api_core.py` (30 collected, 29 run; the skipped one needs `AEGIS_TEST_POSTGRES_URL`). PostgreSQL migration was verified earlier against a `postgres:16` container according to the plan; that PostgreSQL test was skipped in this session's run and not re-executed UNVERIFIED HERE.

10.1 Relational modelling in this project

Timestamps are stored as UTC epoch seconds in `BigInteger` columns so SQLite and PostgreSQL behave identically. JSON columns (JSONB on PostgreSQL) are reserved for evidence and documents such as symptoms, tool calls and report payloads; relational columns hold everything that needs filtering or joining. Foreign keys cascade from `pipeline_runs` so deleting a run removes its data. Identifiers produced by the analytics layer are deterministic per scenario, so stored ids are made run-scoped (`R<run>-ANOM-...`); incident ids are allocated globally (`INC-2026-001`, `INC-2026-002`, and so on).



Arrows run from parent (PK) to child (FK). Not drawn: pipeline_runs -> incidents (run_id); anomalies -> incident_anomalies; root_cause_analyses -> reports (rca_id). audit_logs and security_findings are standalone tables (no foreign keys).

Figure 17. Entity-relationship diagram of the 19 tables (from `app/models/`)

10.2 SQLAlchemy, Alembic and PostgreSQL

SQLAlchemy 2 maps Python classes to tables (`Mapped[...]` annotations; a naming convention gives deterministic constraint names). Alembic holds one migration, `0001_initial_schema`, applied by `alembic upgrade head` (in the Docker entrypoint and the demo). The same migration runs on SQLite (offline demo, tests) and PostgreSQL 16 (Docker). On PostgreSQL, audit appends take an advisory lock so the hash chain cannot fork.

10.3 FastAPI, Pydantic and validation

FastAPI builds the routes and an OpenAPI description served at `/openapi.json` and `/docs`. Request bodies are Pydantic models with `extra="forbid"`: an unknown field is a 422, not silently ignored. Path and query parameters

carry regular-expression patterns (incident ids, report ids, cell ids, metric names) and numeric bounds, for example `minutes` must be 60 to 1440. FastAPI's own documentation describes its `fastapi.security` utilities that integrate with the interactive docs [R3]; Aegis-Net uses an `HTTPBearer` dependency wrapped in `require_role`.

Status codes used: 200 or 201 for success, 401 invalid or missing token, 403 insufficient role or separation-of-duties violation, 404 unknown id, 409 invalid state (for example executing an unapproved action), 413 body too large, 422 validation failure, 429 rate limit, and a generic 500 with only a request id.

10.4 Endpoint reference

Table 9. REST endpoints (verified against `backend/app/api/*.py`; 26 routes plus `/metrics`)

Method and path	Minimum role	Purpose
GET `/api/health`	public	Status, database, C library and provider names; no secrets
POST `/api/pipeline/run`	operator	Run a pipeline. Body: `{"scenario":"seeded"|"healthy","seed":7,"minutes":360,"rollback_minute":null}`. Returns a run summary (201)
GET `/api/pipeline/runs`	viewer	List runs
GET `/api/calls/statistics`	viewer	C-library statistics for a run (`?run_id`, default latest)
GET `/api/calls/anomalies`	viewer	C-library anomalies
GET `/api/network/metrics`	viewer	KPI series (`metric`, `cell_id`, `start`, `end`, `run_id`)
GET `/api/network/anomalies`	viewer	Detected anomalies (`cell_id`, `metric`, `severity`)
GET `/api/network/overview`	viewer	Health score, KPI current versus baseline, per-cell status
GET `/api/incidents`	viewer	Incident summaries (`status`, `severity`)
GET `/api/incidents/{id}`	viewer	Detail with timeline, cells, symptoms, hypotheses, caveats
POST `/api/incidents/{id}/investigate`	operator	Run the RCA (201)
GET `/api/incidents/{id}/rca`	viewer	Latest RCA with the explanation chain
POST `/api/reports/generate`	operator	Body `{"incident_id":"INC-2026-001"}`; stores a report (201)
GET `/api/reports`	viewer	List (`incident_id`)
GET `/api/reports/{report_id}`	viewer	`?format=json|markdown|html`
GET `/api/agent/status`	viewer	Policy table, pending approvals
GET `/api/agent/actions`	viewer	Actions (`run_id`)
GET `/api/agent/runs`, `/api/agent/runs/{run_id}`	viewer	Runs and detail
POST `/api/agent/investigate`	operator	Body `{"scenario":"production_anomaly","incident_id":null}` (201)
POST `/api/agent/approve/{action_id}`	approver	Body `{"decision":"approve"|"deny","comment":""}`
POST `/api/agent/execute/{action_id}`	operator (admin for HIGH_RISK)	Execute an approved action
GET `/api/deployments`	viewer	Deployment records
GET `/api/security/findings`	viewer	Synthetic security findings
GET `/api/audit`	operator	Audit rows (`action`, `limit`, `offset`)
GET `/api/audit/verify`	admin	Hash-chain verification: `{"valid":true,"rows":24,"first_invalid_id":null,"reason":null}`
GET `/metrics`	none (not proxied)	Prometheus text metrics

An example call, using a token from the git-ignored `.aegis-tokens` file (never printed here):

```
curl -s -H "Authorization: Bearer $VIEWER_TOKEN" http://127.0.0.1:8000/api/incidents
```

10.5 Pipeline execution

`services/pipeline.py` runs simulate, CDR generation, C analysis, detection, correlation and storage, then persists metrics in batches of 20,000 rows. In the demo this takes seconds and runs synchronously inside the request; there is no job queue (Chapter 20).

11 Authentication, Security, and Auditability

> **Implementation status.** VERIFIED `backend/app/security/`, `api/deps.py`, `agents/`; `test_security.py` (22 tests, 21 run; the live-Redis test is skipped without `AEGIS_TEST_REDIS_URL`), `test_api_core.py`, `test_agent.py`.

11.1 Authentication versus authorisation

Authentication asks who you are; *authorisation* asks what you may do. Aegis-Net authenticates with bearer tokens: configured only as SHA-256 hashes in `AEGIS_API_TOKENS` (format `name:role:sha256`), compared in constant time against every entry (no early exit) with `hmac.compare_digest`. Raw tokens are generated by `make env`, written to the git-ignored `.aegis-tokens` file with mode 600, and shown only there. Tokens are accepted only in the `Authorization: Bearer` header, not in query strings, cookies or `Basic` auth (tested).

Authorisation uses four ordered roles, each including the lower ones: viewer (1), operator (2), approver (3), admin (4) (`config.Role` is an `IntEnum`).

Table 10. Role-permission matrix derived from `require_role` in `api/deps.py` and `agents/policy.py`

Capability	Viewer	Operator	Approver	Admin
Read dashboards, incidents, RCA, reports, agent state	yes	yes	yes	yes
Run pipeline, investigate, generate reports, start agent runs	no	yes	yes	yes
Read the audit log	no	yes	yes	yes
Approve or deny agent actions (not own runs)	no	no	yes	yes
Execute approved REQUIRES_APPROVAL actions	no	yes	yes	yes
Execute approved HIGH_RISK actions (when the flag is on)	no	no	no	yes
Verify the audit hash chain	no	no	no	yes

An approver is not blocked from starting runs, because roles are cumulative; the *separation of duties* rule instead blocks anyone approving an action of a run they started (`principal.name == run.started_by`).



Figure 18. Dashboard behaviour when a viewer opens the Audit page

Figure 18 shows a signed-in *viewer* pressing "Verify chain": the page reports "Verification unavailable: requires role admin or higher" and "Error: requires role operator or higher" for the list. The control is visible but the server refused the calls. This is the right place for enforcement; hiding buttons would be cosmetic.

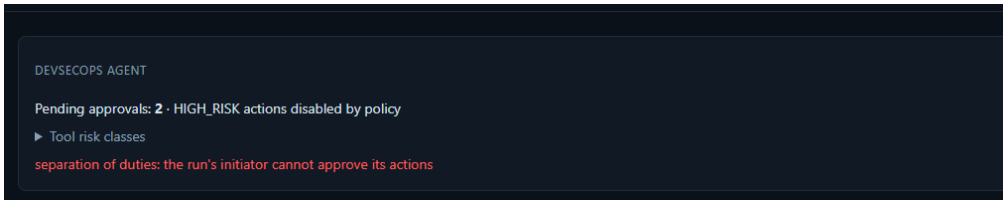


Figure 19. Separation of duties message in the agent panel

11.2 Approval gates

The policy (`agents/policy.py`) classifies each tool as READ_ONLY, LOW_RISK, REQUIRES_APPROVAL, HIGH_RISK or BLOCKED and returns allow, needs_approval or deny; Chapter 15 details the flow. The decisive point is that `execute_action` re-checks every rule immediately before running, so the API route is only a second line of defence.

> **Note.** A policy written in a document is only a promise. A policy enforced in application code, re-checked at the moment of action and covered by tests that try to bypass it, is a control. Aegis-Net's agent policy is the latter: the repository's tests include a test that an unapproved action cannot execute via the service and one that the initiator cannot approve their own action.

11.3 Request identifiers, logging and audit

Every response carries `X-Request-ID` (a valid incoming value is echoed, otherwise generated); JSON access-log lines record method, route *template*, status, duration and actor. A log-record factory stamps `request_id`, `incident_id` and `agent_run_id` onto records. Tokens and API keys are never logged (tested).

The **audit log** is append-only. Each row stores `prev_hash` and `hash`, where the hash is the SHA-256 of a canonical JSON of the row's content plus the previous hash, starting from 64 zeros:

```
def _digest(prev_hash: str, fields: dict[str, Any]) -> str:
    canonical = json.dumps({"prev": prev_hash, **fields}, sort_keys=True, separators=(",", ":"),
                           default=str)
    return hashlib.sha256(canonical.encode("utf-8")).hexdigest()
```

`verify_chain` walks the rows in order and reports the first row whose `prev_hash` or content hash does not match. Editing or deleting a row therefore invalidates every later link. The application code only ever inserts audit rows (the other uses of the table are reads for listing and verification), but the database itself has no trigger or permission blocking UPDATE or DELETE, so "append-only" is a property of the code plus the tamper evidence of the chain, not of the database. A caveat: a database administrator who recomputes all later hashes could forge a consistent chain, so for stronger guarantees the head hash would need external anchoring, which is not implemented **NOT IMPLEMENTED**.

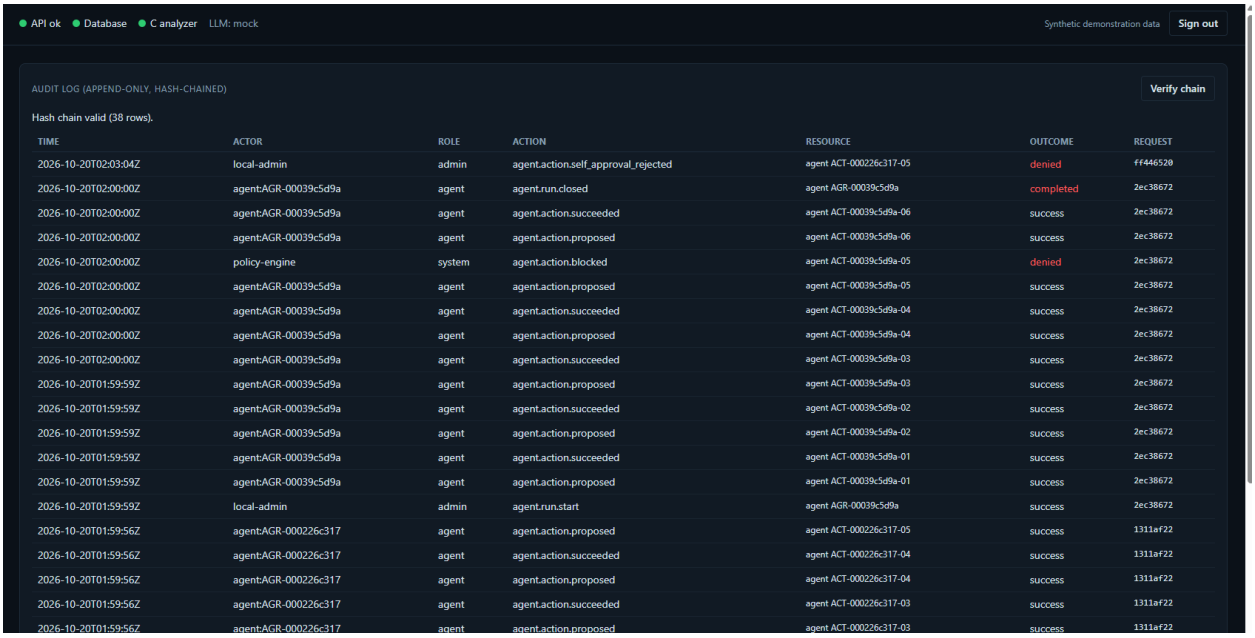


Figure 20. Audit Log page: hash chain valid (38 rows)

Figure 20 shows rows for agent runs (`agent.run.start`, `agent.action.proposed`, `agent.action.succeeded`), a policy-engine block (`agent.action.blocked`, outcome denied), and a rejected self-approval by `local-admin`

(`agent.action.self_approval_rejected`, denied). The request column groups rows made in one request. Denied authentication attempts are audited too (`auth.denied`, `auth.forbidden`).

11.4 Input validation, limits and rate limiting

Pydantic `extra="forbid"` bodies; patterns on ids, metrics and cells; a 1 MB request limit enforced for declared and streamed bodies (413); security headers (`nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, a restrictive Content-Security-Policy on `/api/*`); and a fixed per-minute rate-limit window keyed by a hash prefix of the Authorization header or the client IP, using Redis when `REDIS_URL` is set and an in-process limiter otherwise (default 120 per minute; the Docker stack uses 600).

11.5 Prompt injection and untrusted log content

Logs, security events and report text may contain attacker-controlled strings. Controls in the code: the RCA system prompt tells the model that tool results are data and any instructions inside them must be ignored; tools are read-only, incident-scoped and size-limited, so even a successful injection could only change what the model *says*, not what the system *does*; the model's answer must validate against a strict schema and cite real tool calls; and no LLM output ever becomes a shell command or tool choice in the agent. HTML reports are autoescaped and Markdown cells escape `|`, `\`, `<`, `>` and newlines (tested with injected `<script>` payloads), and the dashboard renders HTML reports in a sandboxed iframe.

> **Limitation.** A prompt in the system message is not a guarantee: a live model could still be misled into a wrong conclusion. The mitigation is structural (read-only tools, validation, human approval), not the prompt wording. Live-model behaviour against injected content was not tested UNVERIFIED HERE.

11.6 Safe command execution and secrets

Agent commands run only through `CommandRunner` (Chapter 15): fixed executables, `shell=False` enforced by an AST test, argument allowlists, a confined working directory, a scrubbed environment, a timeout and truncated output. Secrets are read from environment variables only; `Settings` marks keys and token hashes `repr=False`; `.env`, `.aegis-tokens` and virtual environments are git- and docker-ignored; no secret value appears in this handbook, the slides or the screenshots (checked).

12 LLM Provider Abstraction

> **Implementation status.** VERIFIED interface, mock provider, provider selection, schema validation, and request-shape tests with fake SDK clients. UNVERIFIED HERE Anthropic, OpenAI and local providers were never called live: no API key was available, and the opt-in live test (`AEGIS_RUN_LIVE_LLM_TESTS=1`) was skipped in this session's run.

12.1 Basics

An LLM receives a conversation and, when given *tool schemas* (JSON descriptions of functions with typed arguments), may reply with a request to call one. **Structured output** means the final answer must be a JSON object matching a schema. A **JSON schema** describes the allowed fields and types; **Pydantic** validates data against Python models and can export a JSON schema from them.

12.2 Provider abstraction

`backend/app/llm/base.py` defines a provider-neutral interface: messages (`UserMessage`, `AssistantMessage` carrying the provider's raw content, `ToolResultsMessage`), `ToolSpec`, `ToolCall`, `ToolResult`, `LLMResponse`, and an `LLMProvider` with two methods: `generate(system, messages)` for plain text and `tool_call(system, messages, tools)` for one turn that may request tools. Providers translate to their wire format and *never execute tools*. Errors are normalised to `LLMRefusalError`, `LLMRateLimitError` and `LLMUnavailableError` (both retryable) and `LLMConfigurationError`. The abstraction matters because the RCA engine, reports and tests can then run unchanged against a deterministic mock offline and against a real model online.

Table 11. Providers as implemented

`LLM_PROVIDER`	Class	Behaviour	Verification
`mock` (default)	`MockProvider`	Deterministic; delegates `tool_call` to a rule-based investigator and `generate` to a template; not an LLM	VERIFIED used by tests, demo, screenshots
`anthropic`	`AnthropicProvider`	Official SDK, lazy import; default model `claude-opus-5-5`; effort set explicitly (`high`); every tool `strict`; tool choice auto; server-side refusal fallback beta; refusals checked first; assistant content echoed unchanged; SDK errors mapped	UNVERIFIED HERE request shape tested with fakes
`openai`	`OpenAIProvider`	Chat Completions with `strict` function tools; `MODEL_NAME` is required; malformed tool-call JSON surfaces as an invalid-arguments error	UNVERIFIED HERE
`local`	`OpenAIProvider(base_url=LOCAL_LLM_BASE_URL)`	Any OpenAI-compatible server	UNVERIFIED HERE

Selection happens in `factory.build_provider`. If the chosen provider lacks its key, URL or SDK, the factory returns the mock and a `fallback_reason` that the API exposes; an unknown name also falls back. This keeps the system usable offline, but it means a misconfigured live provider silently becomes the mock unless someone reads the reason, so `/api/health` reports the provider name.

12.3 Configuration

Environment variables (no values here): `LLM_PROVIDER`, `MODEL_NAME`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `LOCAL_LLM_BASE_URL`. Vendor SDKs are optional installs (`backend/requirements-llm.txt`; included in the Docker image).

12.4 Sequence of an investigation

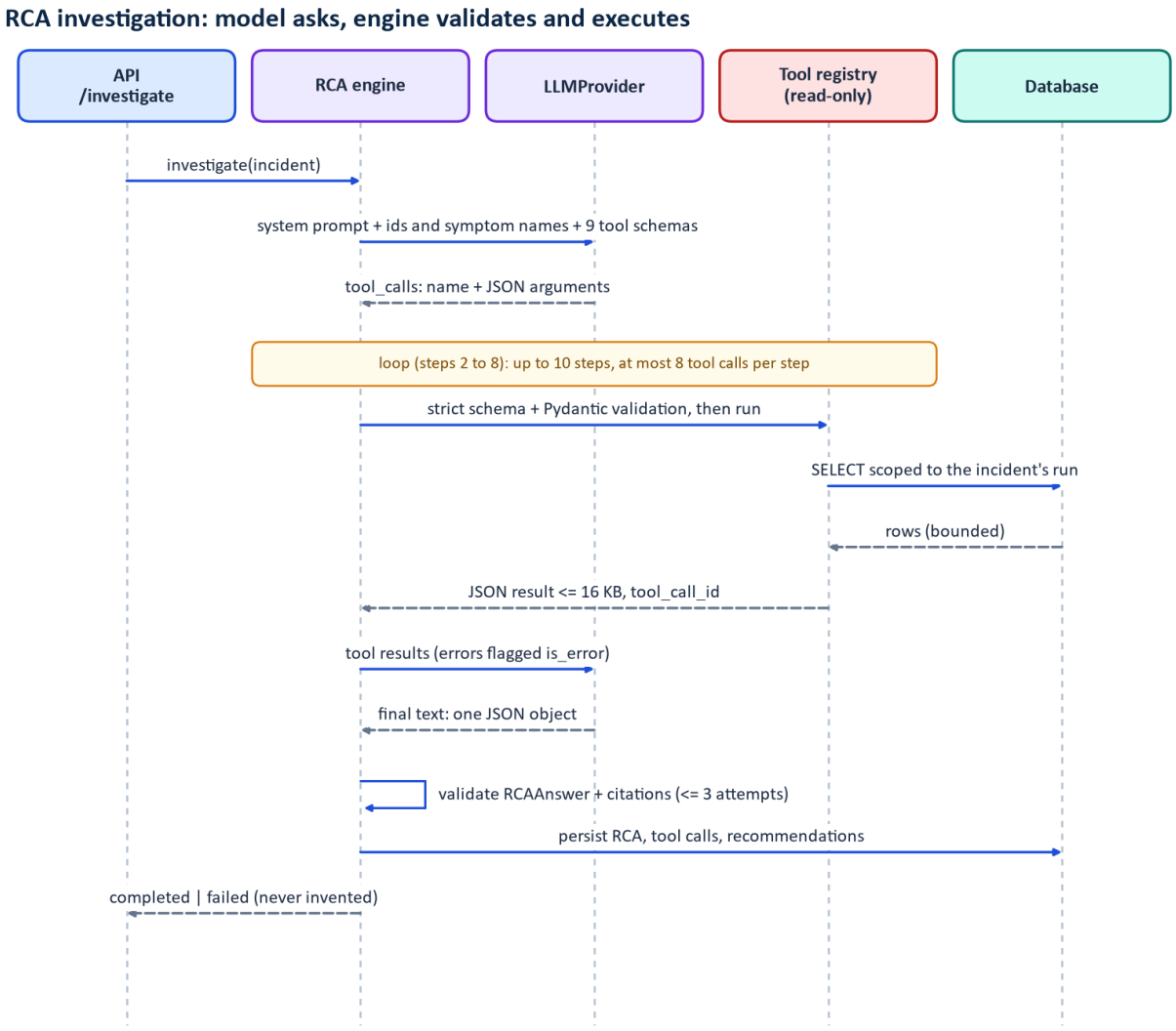


Figure 21. RCA tool-calling sequence

Figure 21 is the loop of `rca/engine.py`. The opening prompt contains only identifiers and symptom names, never raw data. The model requests tools; the engine looks each name up in the allowlisted registry, validates the arguments twice (the strict JSON schema sent to the model, and a Pydantic model with `extra="forbid"` and strict types), runs the tool, enforces a 16,000-character result limit and returns the result with its `tool_call_id`. Unknown tools, invalid arguments and tool errors go back to the model as `is_error` results and are recorded, never executed. Failures from the provider end the investigation as `failed` with the reason.

12.5 Deterministic testing with the mock

`rca/mock_investigator.py` follows a fixed plan through the *same* read-only tools and builds its answer only from the results it received, citing their ids; the answer then passes the *same* validation as a real model's. That makes the validation, citation and persistence paths testable offline. It also means mock RCA output demonstrates the pipeline, not language-model quality. In the seeded run the mock made 12 tool calls.

> **Warning.** The mock provider is rule-based. Everything labelled "mock" in a screenshot (the header shows `LLM: mock`, the RCA page `mock/mock-deterministic-v1`) was produced by deterministic code, not by a language model. Do not read it as evidence of how a live model would reason.

13 Root-Cause Analysis Engine

> **Implementation status.** VERIFIED with the mock provider; live providers UNVERIFIED HERE.

13.1 Workflow

1. **Incident selection.** `POST /api/incidents/{id}/investigate` (operator) loads the incident and moves it to `investigating`.
2. **Evidence collection** through nine read-only tools (`rca/tools.py`):

Table 12. RCA tools

Tool	Returns
`get_call_statistics`	Overall statistics from the C analyser with per-cell figures
`get_cell_statistics`	Call statistics, call anomalies and metric anomalies for one cell
`get_metric_history`	One KPI on one cell, downsampled to at most 120 points
`get_anomalies`	Metric anomalies overlapping a time range
`get_related_events`	The incident's timeline, correlation hypotheses (answers only for the incident under investigation)
`get_recent_deployments`	Recent packet-gateway deployments
`get_security_events`	IDS events aggregated by category and target
`get_logs`	Log entries for one known service
`compare_baseline`	KPI mean in a range versus the preceding 60 minutes, per cell

Tool limits: known metric names, cell patterns and services only; time spans of at most 24 hours; incident-scoped read-only database access; bounded output.

1. **Hypothesis generation and evaluation.** The answer must contain `facts`, `inferences`, `hypotheses` (cause type, confidence of at most 0.95, supporting and contradicting evidence, an assessment), `recommended_actions` (priority, risk class, optional agent tool) and `verification_steps`.
2. **Citations.** Facts and inferences cite tool-call ids. `validate_citations` rejects an answer that cites a call that did not happen, a fact with no *successful* call, or a leading hypothesis without supporting evidence from a successful call. The root cause must be the highest-confidence hypothesis and its confidence must match.
3. **Limits and retries.** At most 10 steps, 8 tool calls per step and 3 answer attempts; a rejected answer receives feedback; after the last attempt the RCA is stored as `failed` and no cause is invented.

13.2 Facts, inferences, hypotheses, recommendations

Observed measurements are *facts* (each with a tool-call id). Relationships derived from facts are *inferences*. Candidate causes with confidences are *hypotheses*, and the headline is always labelled a hypothesis. *Recommendations* carry a risk class. The dashboard shows this explicit chain: observed evidence, correlated events, candidate hypotheses and evaluation, most likely root cause, confidence, recommended remediation.

13.3 Worked example from the seeded incident



Figure 22. Root Cause Analysis page for INC-2026-001 (mock provider)

Figure 22 presents: "Deployment of packet-gateway 2.4.1 to pgw-1 introduced a regression (resource growth first)", confidence 85 %, labelled "hypothesis, not a confirmed fact". Under *observed evidence* each bullet ends with a tool-call id such as `[mock-3-1]`, for example "memory_utilization on 4 affected cell(s): window mean 95.15 vs baseline 57.15; largest change on CELL-103 (67.4%)". Others state that packet-gateway 2.4.1 was deployed to pgw-1 at 14:05, 8 minutes before the first anomaly, that other deployments targeted other nodes, that 9 security events in the window targeted none of the affected cells, that 41 warning or error log entries from packet-gateway began at 14:47 with a GC pause, and that the C analysis found a 5.0 % average drop rate across the affected cells against 2.1 % network-wide. The hypothesis table gives deployment_regression 85 %, memory_exhaustion 34 % ("describes the mechanism, but the deployment is the more specific trigger") and security_attack 5 % ("0 security events targeted the affected elements"). Recommended remediation begins with rolling back packet-gateway on pgw-1 from 2.4.1 to 2.4.0, marked HIGH_RISK.

13.4 Presenting conclusions without exposing hidden reasoning

Provider "thinking" blocks are never stored or shown. What is stored is the structured facts, inferences, hypotheses, evidence references and recommendations plus the list of tool calls with arguments and results. That gives an auditor the evidence trail without a transcript of private reasoning, which also avoids presenting unverifiable narrative as evidence.

14 Report Generation

> **Implementation status.** VERIFIED JSON, Markdown and HTML; 11 tests in `test_reporting.py`. NOT IMPLEMENTED PDF output.

14.1 Data model and numeric consistency

`ReportDocument` (`reporting/document.py`) is a Pydantic model with `extra="forbid"` and sections: KPI analysis, anomaly analysis, correlation analysis, root cause, impact, recommendations, verification steps, confidence, evidence references and timeline. `build_report` computes every figure from the database: for each KPI, the mean of the affected cells over the hour before onset (baseline) against the incident window, plus the worst value in the bad direction and the percentage change. Users connected is the sum over cells of mean `active_users`; call counts come from the C analysis and cover the whole analysed log for the affected cells (stated in the text, not just the incident window).

14.2 Narrative and template fallback

Only the executive summary is generated prose. It comes from `LLMProvider.generate` (the mock uses a deterministic template) and is accepted only if every number in it appears in the JSON context the model was given (`_narrative_is_grounded`); otherwise the template is used and `narrative_provider` records the rejection. Provider errors also fall back to the template.

14.3 Renderers and escaping

Jinja2 templates render Markdown and HTML from the same document. The HTML environment autoescapes, `StrictUndefined` makes a missing field an error, Markdown table cells escape special characters, and the HTML endpoint sends `Content-Security-Policy: default-src 'none'; style-src 'unsafe-inline'` and `X-Content-Type-Options: nosniff`.

14.4 An actual excerpt

The excerpt below is from the report written by `make demo` for this handbook (`RPT-INC-2026-001-03`). It covers the window up to the simulated rollback, so its means are lower than the report in the screenshot, which covers the full unmitigated window.

```
## KPI Analysis
| KPI | Baseline (1 h before) | Incident mean | Worst | Change |
| latency_ms (ms) | 22.331 | 48.059 | 418.673 | 115.2% |
| memory_utilization (%) | 57.148 | 81.375 | 99.281 | 42.4% |
Symptom onset sequence (temporal order, not proof of causation):
- memory_utilization -> cpu_utilization (+18 min)
- cpu_utilization -> latency_ms (+9 min)
Correlation hypotheses (ranked, not proven):
- deployment_regression - score 0.8947
- memory_exhaustion - score 0.675
```

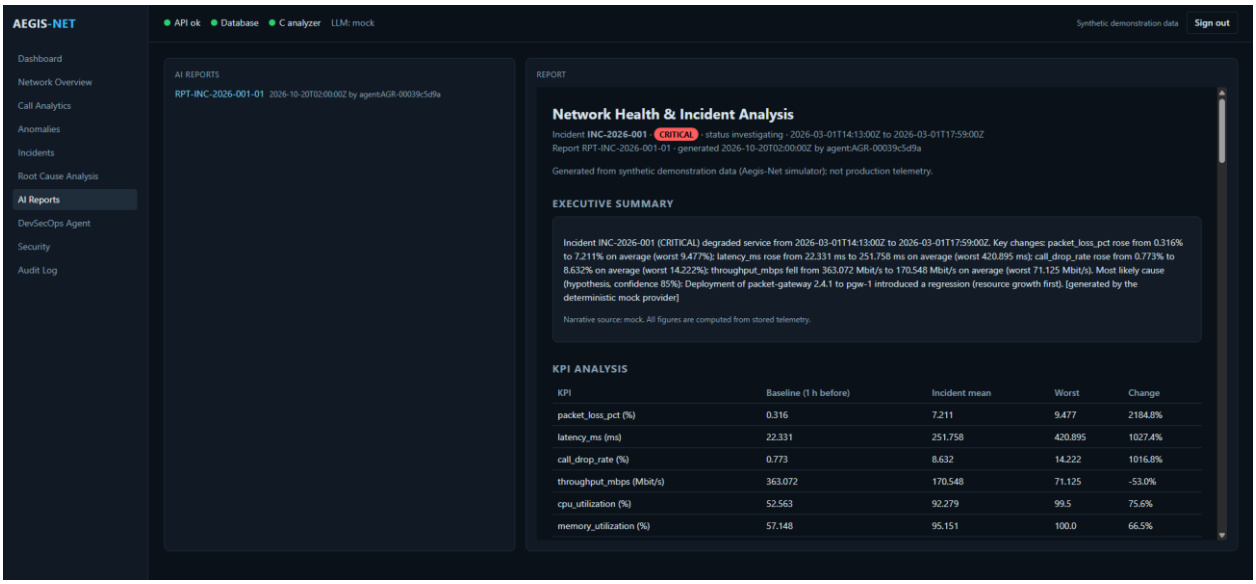


Figure 23. AI Reports page with the executive summary and KPI table

Figure 23 shows the report list (`RPT-INC-2026-001-01`), the synthetic-data notice, the executive summary ("... Most likely cause (hypothesis, confidence 85%) ... [generated by the deterministic mock provider]"), and the KPI table (for example packet loss 0.316 to 7.211 %, worst 9.477 %).

> **Limitation.** No PDF renderer exists. Reports are consumed as JSON, Markdown or HTML. The report's call counts are for the whole call log, and the narrative is only as good as the provider behind it (the mock in all verified runs).

15 Autonomous DevSecOps Agent

Implementation status. VERIFIED `backend/app/agents/` with 36 tests in `test_agent.py`; the offline demo exercises the production-anomaly chain. SIMULATED The rollback is a simulation. NOT IMPLEMENTED `restart_service` has no model behind it and always fails with an explanation.

Warning. "Autonomous" is used in the project's name, but the planner is a deterministic set of rule-based playbooks. The LLM influences only the RCA text. It never chooses an agent tool, an argument or a command. Treat the agent as a *policy-controlled automation*, not as an LLM-driven decision maker.

15.1 The loop

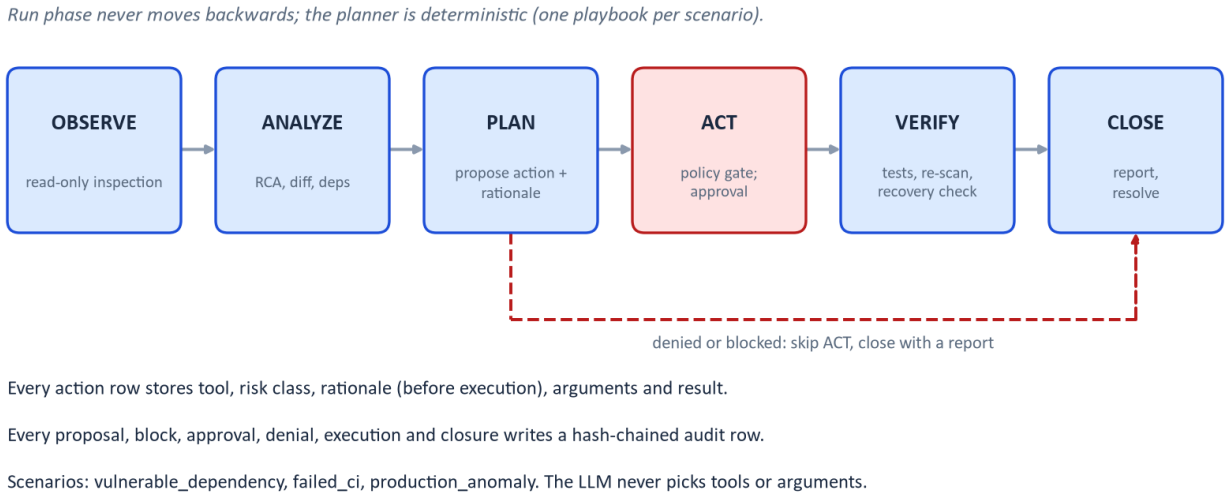


Figure 24. Agent lifecycle

A run moves OBSERVE, ANALYZE, PLAN, ACT, VERIFY, CLOSE and never backwards (`service._advance` raises if the planner attempts it). For each step the planner (`planner.next_action`) is a pure function of the run's action history: it names the tool, its arguments (built from earlier *results*, never from LLM text) and a rationale. `_propose` stores the action with that rationale *before* execution, evaluates the policy, and either executes immediately, parks it as `awaiting_approval`, or marks it `blocked`. A run stops advancing while any action awaits approval. A hard limit of 40 actions per run prevents loops.

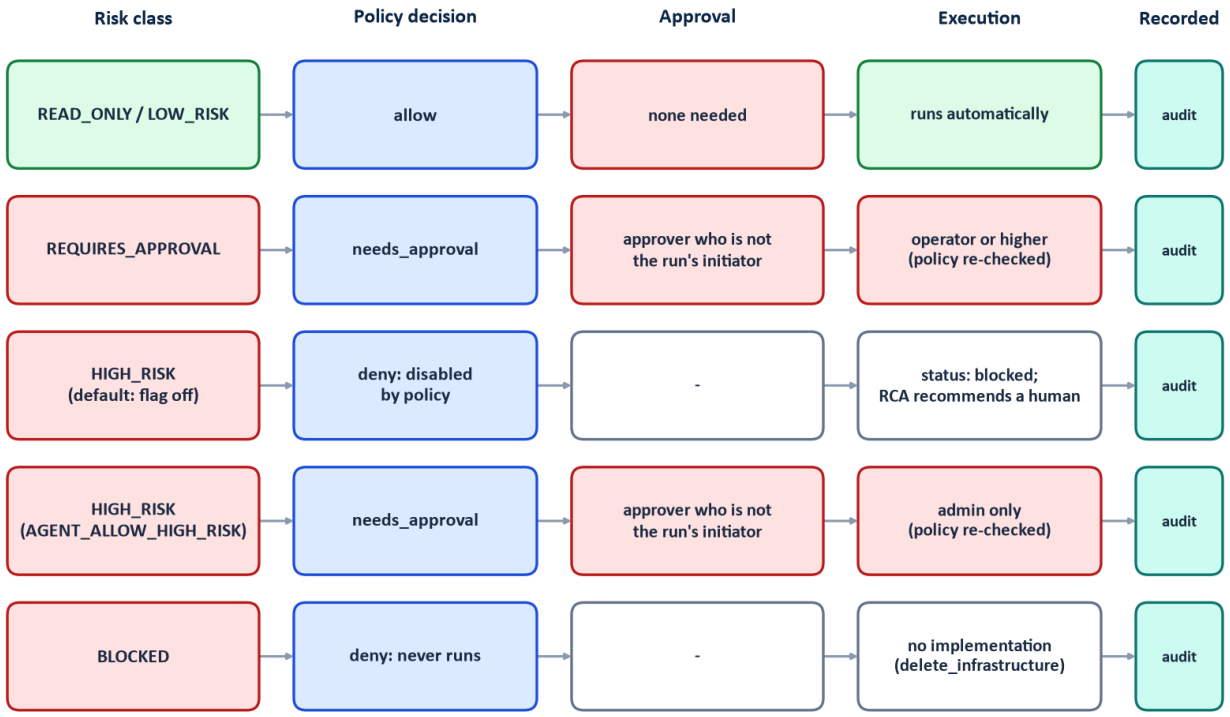
15.2 Tools and risk classes

Table 13. The 20 registered tools (`agents/tools.py`)

Risk class	Tools
READ_ONLY	`inspect_repository`, `inspect_git_diff`, `run_unit_tests`, `run_linter` (pyflakes), `run_tests`, `run_security_scan`, `inspect_dependencies`, `inspect_container`, `inspect_logs`, `inspect_metrics`
LOW_RISK	`create_incident`, `request_rca`, `generate_patch`, `generate_report`, `close_incident`
REQUIRES_APPROVAL	`apply_patch`, `modify_dependency`, `restart_service` (always fails: not modelled)
HIGH_RISK	`rollback_deployment` (simulated)
BLOCKED	`delete_infrastructure` (no implementation)

Every tool has a Pydantic argument model with `extra="forbid"`; the executor validates arguments again at execution time.

15.3 Policy and approval



Source: backend/app/agents/policy.py (evaluate, executor_role); service.py (execute_action re-checks every rule).

Figure 25. Risk classification and approval workflow

Figure 25 shows what `policy.evaluate` and `execute_action` enforce. READ_ONLY and LOW_RISK run automatically. REQUIRES_APPROVAL needs an approver who is not the run's initiator, then an operator or higher to execute. HIGH_RISK is denied (status `blocked`) unless `AGENT_ALLOW_HIGH_RISK=true`; when enabled it still needs approval and an *admin* to execute. BLOCKED never runs. `execute_action` re-evaluates the policy and the executor's role immediately before running, and writes `agent.action.execute_rejected` to the audit log on refusal.

15.4 The sandbox and the disposable workspace

Code scenarios run in a throwaway copy of `demo-workspace/` (a small fictional packet-gateway service with tests) created under `AGENT_WORKSPACE_ROOT/<run>` with its own git history (`workspace.create_workspace`). `CommandRunner` (`agents/runner.py`) enforces:

- only two executables, `python` (this interpreter, `-m pytest` or `-m pyflakes`) and `git` (subcommands `init`, `add`, `commit`, `diff`, `log`, `apply`, `status`, `rev-parse`, `show`; no `-c` overrides);
- argument lists with `shell=False`, each argument checked for NUL bytes and length, a rule an AST test enforces over the codebase;
- the working directory must lie inside the workspace root, and workspace-relative paths are resolved and checked (`inside`);
- an environment scrubbed down to a fixed `PATH`, `HOME` set to the workspace, and git configuration disabled, so no inherited secret or `LD_PRELOAD` leaks in;
- a timeout (default 120 s) and output truncated at 20,000 characters.

15.5 Synthetic advisories

`agents/data/advisories.json` holds two invented advisories with a notice that they are fictional: DEMO-CVE-2026-0001 (netparse below 1.4.3, CRITICAL) and DEMO-CVE-2026-0002 (fastjsonx below 2.0.4, MEDIUM), and a fictional package index. The workspace's `requirements.txt` pins `netparse==1.4.2` and `fastjsonx==2.1.0`. Nothing is downloaded or installed.

15.6 The three scenarios

Table 14. Scenario walk-through (`planner.py`)

Scenario	Steps	What really happens	Approval
`vulnerable_dependency`	scan, record finding, inspect dependencies for the smallest fixed release, `modify_dependency`, run tests, re-scan, report	Edits `requirements.txt` in the sandbox from 1.4.2 to 1.4.3 and re-scans against the synthetic database; never deployed	`modify_dependency` needs approval
`failed_ci`	run tests, read CI log, `inspect_git_diff`, `generate_patch`, `apply_patch`, run tests and linter, report	A staged commit "Optimise checksum carry folding" broke `pgw/checksum.py`; the hypothesis is that the changed module is exercised by a failing test; the patch is a revert of that file's change	`apply_patch` needs approval
`production_anomaly`	`inspect_metrics`, `create_incident` (acknowledge), `request_rca`, `inspect_container`, `rollback_deployment`, `inspect_metrics` on the verification run, `close_incident`, `generate_report`	If the RCA's leading cause is `deployment_regression` and an active deployment preceded the incident, propose a rollback. Execution re-simulates the scenario with a rollback 45 minutes after onset and checks that the last 30 minutes are back near baseline	HIGH_RISK: flag on, approval, admin

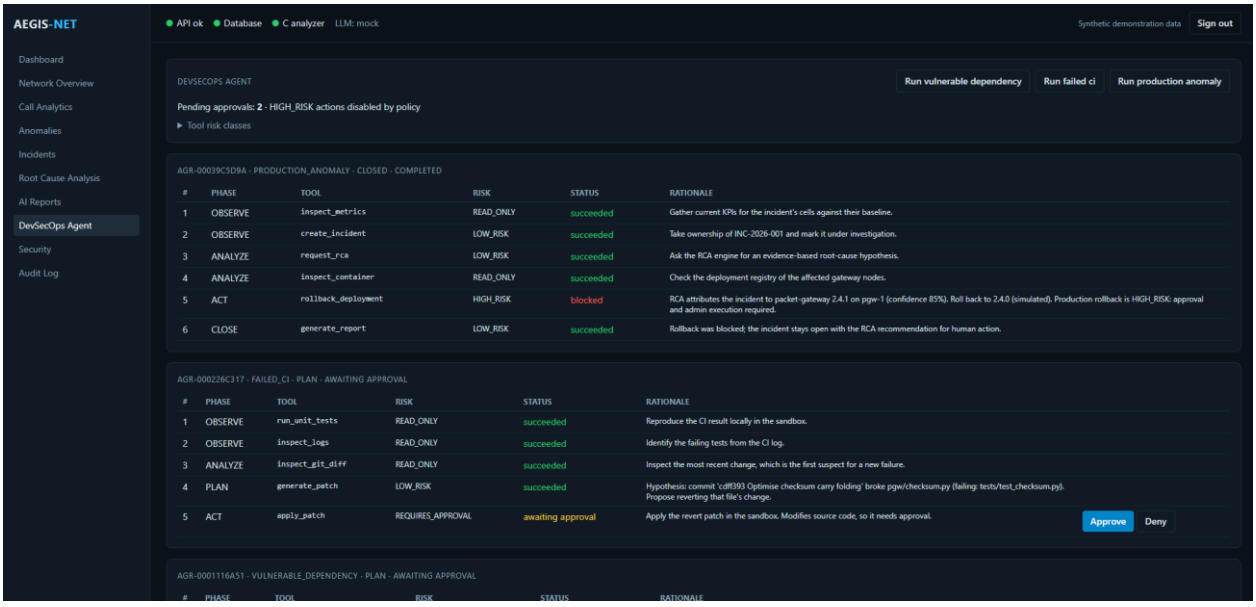


Figure 26. DevSecOps Agent page: one closed run and two runs awaiting approval

Figure 26 shows three runs. `AGR-00039C5D9A` (production_anomaly) is CLOSED: `inspect_metrics`, `create_incident`, `request_rca` and `inspect_container` succeeded, and `rollback_deployment` (HIGH_RISK) shows **blocked** because the header reads "HIGH_RISK actions disabled by policy"; the final `generate_report` rationale says the incident stays open with the RCA recommendation for human action. `AGR-000226C317` (failed_ci) shows `apply_patch` (REQUIRES_APPROVAL) **awaiting approval** with Approve and Deny buttons; its rationale quotes the commit `cdff393 Optimise checksum carry folding` and the failing test file. A third run

(vulnerable_dependency) is also waiting. The header shows "Pending approvals: 2". Each row displays the phase, tool, risk class, status and the rationale recorded before execution.

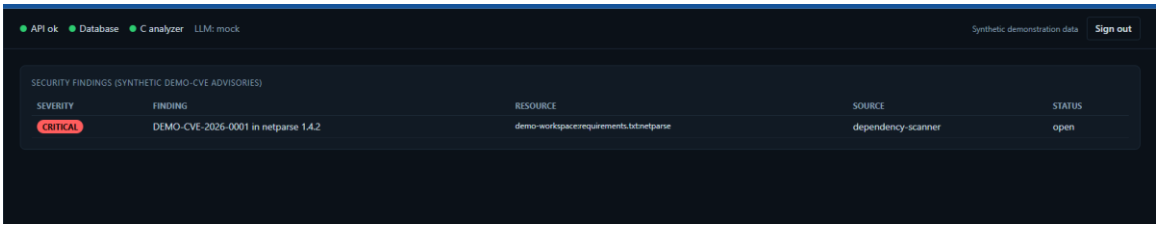


Figure 27. Security page with the synthetic finding

Figure 27 shows `DEMO-CVE-2026-0001 in netparse 1.4.2` (CRITICAL, source dependency-scanner, status open), recorded by the dependency scenario.

15.7 What is real, simulated, approved and blocked

Table 15. Honest inventory of agent behaviour

Behaviour	Real in the sandbox	Simulated	Needs approval	Blocked by default
Run tests, linter, git inspection	yes	no	no	no
Scan dependencies	yes, against fictional advisories	advisory data is synthetic	no	no
Change a dependency pin	yes (sandbox file)	never deployed	yes	no
Generate and apply a revert patch	yes (sandbox git)	no	yes	no
Roll back a production deployment	no	yes (re-simulated telemetry)	yes, plus admin	yes (flag off)
Restart a service	no	not modelled	yes	no (it simply fails)
Delete infrastructure	no	no	n/a	always

Limitation. Because the simulated rollback regenerates telemetry from the same model, "recovery verified" demonstrates the control flow and the verification arithmetic. It cannot show that a real rollback would recover a real service, since the simulator was written to recover when the deployment effect is removed.

16 Frontend and Dashboard

> **Implementation status.** VERIFIED `frontend/` (React 18.3.1, TypeScript 5.6.3 strict, Vite 5.4.11, Tailwind 3.4.15, Recharts 2.13.3, React Router 6.28.0). `make frontend-test` was run for this handbook in a `node:22-alpine` container: type-check, Vitest (2 files, 10 tests passed) and the production build succeeded (exit 0). The recorded browser check of all 11 routes was not re-run.

16.1 Structure

`App.tsx` shows `Login` until a token exists, then a `Layout` (sidebar plus top status bar) with 11 routes: `/` Dashboard, `/network`, `/calls`, `/anomalies`, `/incidents`, `/incidents/:incidentId`, `/rca`, `/reports`, `/agent`, `/security`, `/audit`. Components: `KpiCard`, `TimeSeriesChart` (Recharts), `IncidentTimeline`, `RcaPanel`, `AgentActionPanel`, and shared helpers. `hooks/useApi.ts` is a small data-loading hook (loading, error, reload with cancellation). `types/api.ts` mirrors the Pydantic response models, and `services/api.ts` is the only place that talks to the network, with one function per endpoint (the frontend does not call `/api/deployments`).

16.2 Authentication handling

The token is typed on the sign-in page, validated against the API, and stored in `sessionStorage` (cleared when the tab closes) under one key; every request adds `Authorization: Bearer`. Path parameters are URL-encoded. There are no cookies. A consequence worth noting: any script running in the page origin could read the token, which is why the nginx configuration sets a restrictive Content-Security-Policy and why reports are rendered in a sandboxed iframe.

16.3 Pages and how users interact

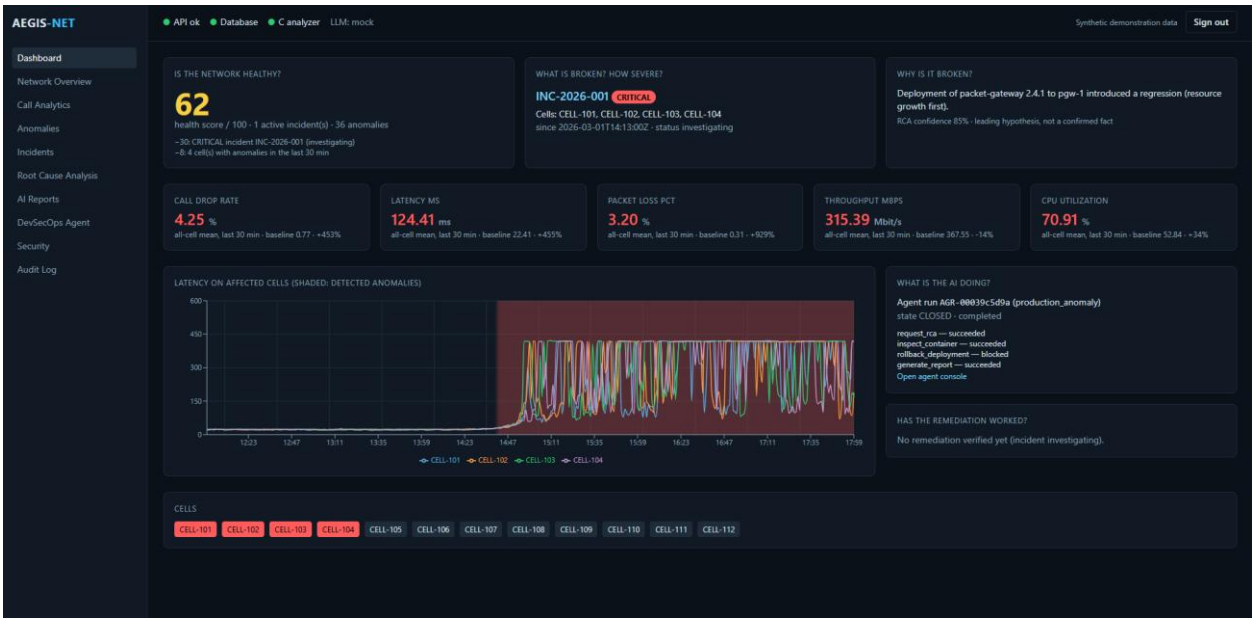


Figure 28. Dashboard

Figure 28 answers six operator questions in panels: *Is the network healthy?* (health score 62 of 100: minus 30 for the critical incident, minus 8 for four cells with anomalies), *What is broken and how severe?* (INC-2026-001, CRITICAL, cells CELL-101 to CELL-104, since 14:13, status investigating), *Why?* (the RCA headline with "RCA confidence 85 %, leading hypothesis, not a confirmed fact"), five KPI cards against baseline, a latency chart of the affected cells with shaded detected anomalies, *What is the AI doing?* (the agent run and its actions) and *Has the remediation worked?* ("No

remediation verified yet (incident investigating)"). The cell strip colours CELL-101 to CELL-104 red. The top bar shows API, database and C analyzer status, `LLM: mock`, and a "Synthetic demonstration data" notice.

The health-score formula is documented in `GET /api/network/overview`: 100 minus 30, 20, 10 or 5 per active CRITICAL, HIGH, MEDIUM or LOW incident, minus 2 per cell with an anomaly in the last 30 minutes, floored at 0. Here 100 - 30 - 8 = 62.

Other pages (screenshots in earlier chapters): Network Overview (Figure 8), Call Analytics (Figure 4), Anomalies (Figure 12), Incidents (Figure 16), RCA (Figure 22), AI Reports (Figure 23), DevSecOps Agent (Figure 26), Security (Figure 27) and Audit Log (Figure 20). The incident detail page (no screenshot supplied) shows the timeline, symptoms, hypotheses with caveats, the RCA chain and actions (investigate, report, agent). Report HTML is displayed in an `<iframe sandbox>` with scripts disabled.

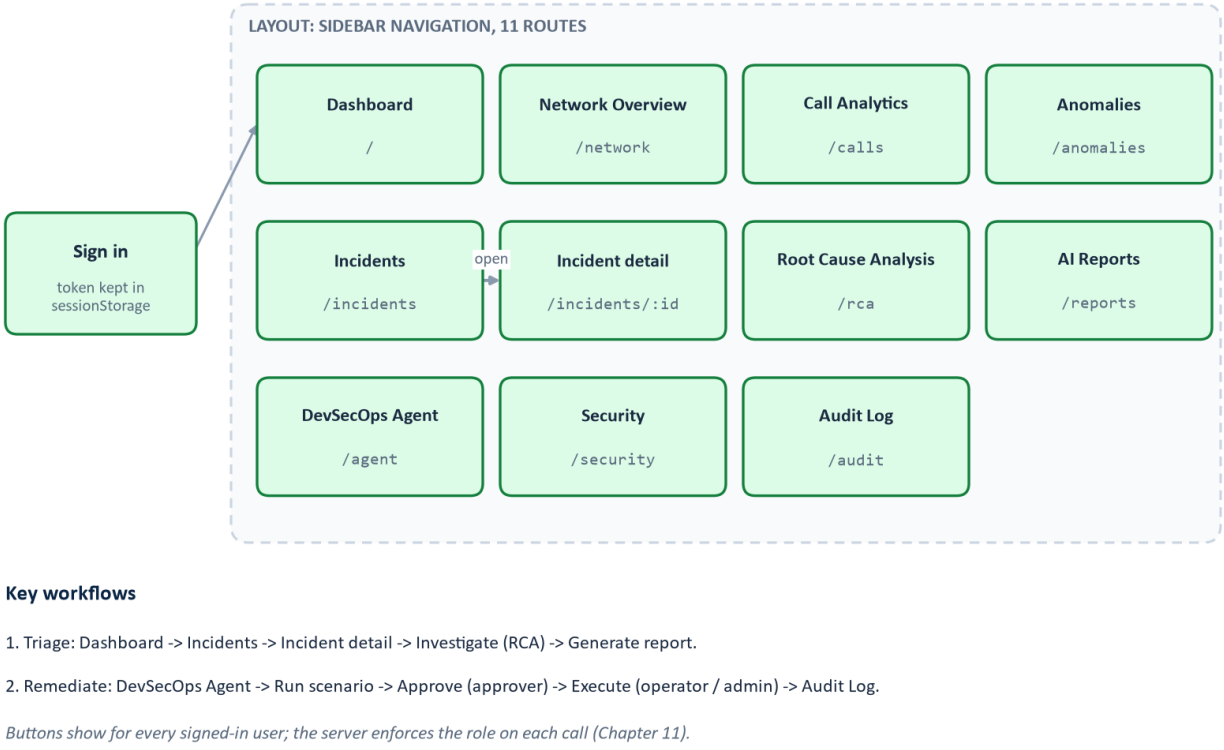


Figure 29. Dashboard navigation and workflows

> **Limitation.** The recorded limitations are: no per-route code splitting, and no browser end-to-end test of the approval flow (the flow is covered by API tests).

17 Docker and Deployment

> **Implementation status.** VERIFIED A clean `--no-cache` smoke build and the four-service start are recorded in `IMPLEMENTATION_PLAN.md` (M11 and the final sweep). They were *not* re-run for this handbook, so for this session the Docker stack is UNVERIFIED HERE. The C build and tests that the image also runs were executed on the host (Chapter 5). The Dockerfile, compose file and smoke script below were read.

17.1 Concepts

A **container image** is an immutable bundle of a filesystem and start command; a **container** is a running instance. A **multi-stage build** has several `FROM` lines, and files are copied between stages so the final image carries only the finished output, not the compilers [R5]. **Docker Compose** declares several containers, their networks, volumes and start order in one file.

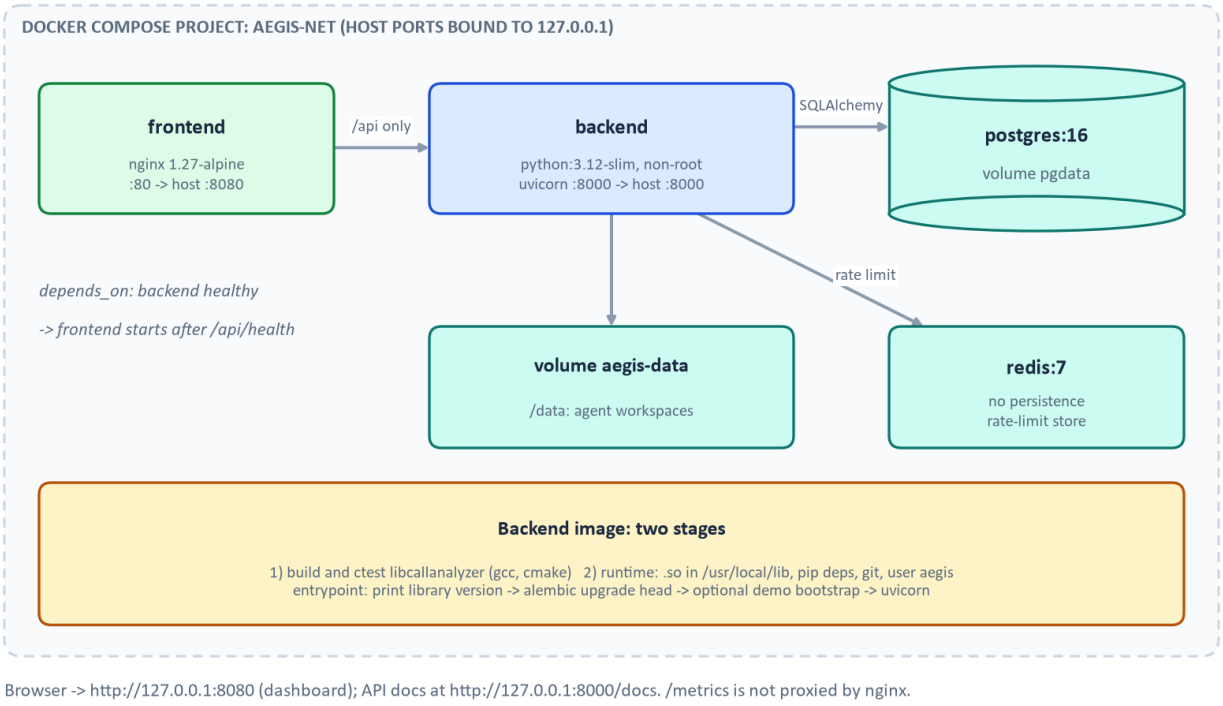


Figure 30. Container topology

17.2 The images

`backend/Dockerfile`: stage 1 (`python:3.12-slim` with gcc, libc headers, cmake, make) configures and builds `libcallanalyzer` in Release mode, **runs ctest during the build** (a failing C test fails the image build) and installs to a prefix. Stage 2 (`python:3.12-slim`) installs git (for the agent sandbox), creates the non-root user `aegis` (uid 10001), copies the library to `/usr/local/lib` and runs `ldconfig`, installs pinned requirements including the optional LLM SDKs, copies `backend/` and `demo-workspace/`, sets `AEGIS_DATA_DIR=/data`, and defines a health check that GETs `/api/health`. The entrypoint prints the library version, runs `alembic upgrade head`, optionally runs `python -m app.bootstrap` (loads the seeded incident into an empty database when `AEGIS_BOOTSTRAP_DEMO=true`; idempotent) and starts uvicorn on port 8000.

`frontend/Dockerfile`: a Node 22 stage runs `npm ci`, **Vitest and the build**, then nginx 1.27-alpine serves `dist`. `nginx.conf` proxies only `/api/` to `backend:8000`, falls back to `index.html` for client-side routes, answers `/healthz`, and sets security headers including a Content-Security-Policy. `/metrics` and `/docs` are not proxied.

17.3 Services, ports, volumes

Table 16. Compose services (`docker-compose.yml`)

Service	Image	Notes
postgres	`postgres:16`	volume `pgdata`; health check `pg_isready`
redis	`redis:7`	`--save "" --appendonly no` (no persistence); `redis-cli ping` health check
backend	built from `backend/Dockerfile`	environment passed explicitly; `depends_on` postgres and redis healthy; published on `127.0.0.1:${BACKEND_PORT:-8000}`; volume `aegis-data` at `/data`
frontend	built from `frontend/`	`depends_on` backend healthy; published on `127.0.0.1:${FRONTEND_PORT:-8080}`

Ports bind to 127.0.0.1 only. `POSTGRES_PASSWORD` and `AEGIS_API_TOKENS` use the `${VAR:?message}` form, so compose refuses to start without them and tells you to run `make env`.

17.4 Commands

Table 17. Commands, from the Makefile and compose file

Command	What it does
`make env`	Writes `.env` (hashed tokens, generated Postgres password) and `.aegis-tokens` (raw tokens, mode 600); refuses to overwrite an existing `.env`
`docker compose up --build` (or `make docker-up`, which adds `-d`)	Builds and starts all four services
`docker compose down` (or `make docker-down`)	Stops the stack and keeps volumes
`make docker-build`	`docker compose build`
`make docker-smoke`	Isolated project `aegis-smoke` on ports 18000 and 18080 with a throwaway env file; builds, waits for health, checks the API, then removes its containers and volumes (`SMOKE_UI_CHECK=1` adds the browser check)
`docker compose down -v`	Standard Docker flag that also removes named volumes (deletes stored data); not run for this handbook

17.5 Troubleshooting

- *"run make env to create .env"*: the required variables are missing; run `make env` once.
- *Port already in use*: set `BACKEND_PORT` or `FRONTEND_PORT` in the environment.
- *Backend never becomes healthy*: `docker compose logs backend`; the entrypoint prints the library version first, then Alembic output. The smoke script prints the last 50 log lines in this case.
- *Agent reports odd test failures*: the plan records an earlier defect in which pytest was missing from the runtime image and the agent misdiagnosed; the image now installs the pinned requirements, so a rebuild (`docker compose build --no-cache backend`) is the first thing to try.
- *Dashboard shows login after a restart*: tokens live in `sessionStorage`; re-enter one from `.aegis-tokens`.
- *Provider shows `mock` although a key was set*: check `/api/health` and the backend log for the fallback reason.

No credentials are published here; no production environment is needed.

18 Testing and Quality Assurance

Implementation status. Results below were produced by commands run for this handbook unless marked "recorded". Complete commands and outputs are in Appendix B.

Table 18. Test layers, what each proves and what it does not

Layer	Evidence in this session	Proves	Does not prove
C unit tests (28 functions)	`make c-test`: 2 of 2 passed	Parsing, statistics, anomaly logic behave on tested inputs	Behaviour on unseen inputs
Sanitizers (ASan, UBSan, LSan)	`make c-test-asan`: 2 of 2 passed	No detected memory errors or leaks on tested paths	Other compilers or platforms
Fault injection	included in the second C test executable	Every allocation failure path returns cleanly	Non-allocation failures
Python unit and integration tests (355 collected)	`make py-test`: **352 passed, 3 skipped** in 207.68 s	Binding contract, simulator, detectors, correlation, RCA, reports, agent, API, security, end-to-end chains	Live LLMs, PostgreSQL and Redis (the three skips)
ctypes ABI tests	within the above (28 plus 61 tests)	Field-by-field layout match, ownership spies	Layout on other ABIs
Numerical reference tests	`tests/reference.py` used by binding tests	C results agree with an independent reference implementation	Statistical validity of thresholds
Static typing	`make py-typecheck`: mypy strict, 86 source files, no issues	Type consistency	Runtime correctness
Frontend	`make frontend-test`: tsc, Vitest 10 of 10, Vite build	Components and pages render fixtures	Real browser flows (recorded Playwright check only)
Docker smoke	recorded in the plan; not re-run	Stack starts and answers	Anything in this session
End-to-end demo	`make demo`: exit 0, 13 steps, audit chain valid (24 rows)	The whole chain through the real API in-process	Production behaviour

Three tests were skipped by design: PostgreSQL migration and pipeline (`AEGIS_TEST_POSTGRES_URL`), live Redis rate limiter (`AEGIS_TEST_REDIS_URL`), and live Claude RCA (`AEGIS_RUN_LIVE_LLM_TESTS=1` with a key). The ASan variant of the Python suite (`make py-test-asan`) was not re-run for this handbook; the plan records 345 passed. Mutation testing in this project means hand-applied mutations in a scratch copy, recorded in the plan (for example six gate mutations in the agent policy, all detected). There is no mutation-testing tool in the repository, so those results are recorded rather than reproducible by command.

Limitation. Passing tests show that the code does what the tests assert. The simulator, thresholds and tests were written together, so the detection results measure internal consistency, not field accuracy.

19 How to Run and Demonstrate Aegis-Net

19.1 Prerequisites and repository map

Linux (the plan records Ubuntu under WSL2), gcc and CMake, Python 3.12 or newer, and Docker for the full stack and the dashboard build. Node is not needed on the host. Key directories: `c-library/`, `backend/` (`app/`, `call_analyzer/`, `alembic/`, `tests/`), `frontend/`, `scripts/`, `demo-workspace/`, `docs/`, `data/` (generated, git-ignored), `reports/`.

19.2 Environment

```
make c-build     # build libcallanalyzer
make py-venv     # backend/.venv with pinned dependencies
```

Environment variables (no values): `DATABASE_URL`, `AEGIS_API_TOKENS`, `LLM_PROVIDER`, `MODEL_NAME`, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `LOCAL_LLM_BASE_URL`, `REDIS_URL`, `AEGIS_RATE_LIMIT_PER_MINUTE`, `AEGIS_MAX_REQUEST_BYTES`, `AGENT_ALLOW_HIGH_RISK`, `AGENT_WORKSPACE_ROOT`, `LOG_LEVEL`, plus `AEGIS_DATA_DIR`, `AEGIS_BOOTSTRAP_DEMO` (Docker), `POSTGRES_PASSWORD`, `BACKEND_PORT`, `FRONTEND_PORT`.

19.3 Offline demo (no key, no Docker)

```
make demo
```

It builds the library, then runs `scripts/demo.py`: a fresh SQLite database, the mock provider, in-process API calls with role-scoped tokens generated on the fly, ending with `DEMO COMPLETED`. It enables high-risk actions for itself only. It writes a report to `reports/` (git-ignored). This ran for this handbook (exit 0). Everything in the demo works without an LLM API key.

19.4 Docker with the dashboard

```
make env                      # once
docker compose up --build     # dashboard http://127.0.0.1:8080, API docs http://127.0.0.1:8000/docs
```

Sign in with a token from `.aegis-tokens`: use the *viewer* token to explore read-only, *operator* to investigate and report, *approver* to approve, *admin* to execute high-risk actions and verify the audit chain. Note the stack starts with `AGENT_ALLOW_HIGH_RISK=false`, so a production rollback shows as blocked until you set it to `true` in the environment and restart.

19.5 A guided walk-through

1. **Dashboard.** Read the six panels (Figure 28); note the health score and the incident.
2. **Incidents, then the incident row.** Open INC-2026-001 and read the timeline and hypotheses.
3. **Investigate** (operator). The RCA page shows evidence with tool-call ids (Figure 22).
4. **Generate a report** (operator) and open it under AI Reports.
5. **DevSecOps Agent.** Start `production_anomaly`. With the flag off the rollback is blocked (Figure 26); start `failed_ci`, then approve with a *different* user than the one who started it (the same user gets the separation-of-duties message) and execute.
6. **Audit Log.** Press "Verify chain" as admin; read the rows (Figure 20).
7. **Stop.** `docker compose down`.

19.6 Tests

`make test` runs C, sanitizer C, Python, sanitizer Python, mypy and the frontend suite; `make container-test` repeats C and Python in `python:3.12-slim`. Opt-in tests need the environment variables of Chapter 18.

19.7 Live providers

Set `LLM_PROVIDER` to `anthropic`, `openai` or `local`, give the matching key or URL (and `MODEL_NAME` for openai and local), and install `backend/requirements-llm.txt` for a non-Docker run. Without these the system falls back to the mock and reports why. This path is untested live **UNVERIFIED HERE**.

19.8 Common failures

Library not found: run `make c-build` or set `CALL_ANALYZER_LIB` to an absolute path. *401*: wrong or missing token. *403 on approve*: you started the run, or your role is below approver. *409 on execute*: the action is not approved yet. *429*: rate limit; wait a minute.

20 Limitations, Trade-offs, and Future Work

20.1 Honest assessment

- **Synthetic versus real data.** All telemetry, incidents, CDRs, deployments and advisories are generated. Thresholds, floors and the false-positive bound were tuned on this simulator and generic synthetic series. Nothing here shows performance on real traffic.
- **Circular validation.** The simulator, the detectors' thresholds, the correlation graph and the tests share assumptions. The causal graph used for scoring mirrors the simulator's structure, so agreement is expected.
- **Unverified provider integrations.** Anthropic, OpenAI and local paths have never touched a live API; the mock is rule-based.
- **Optional algorithms not built.** Isolation Forest, DBSCAN; also no seasonal baseline, so a real daily cycle would raise false positives until the baseline learns it.
- **Deferred.** PDF reports; OpenTelemetry and Grafana.
- **Simulated rollback and narrow agent.** Patches are limited to reverting the last commit; `restart_service` is unmodelled.
- **Correlation limits.** Caller and per-call C anomalies are not turned into incidents; two independent faults on the same node with near-simultaneous onset would merge; correlation is not causation.
- **Performance unknown.** The pipeline runs synchronously in the request (a few seconds for 360 minutes of 12 cells). No load testing was performed.
- **Security gaps for production.** No TLS termination, no external identity provider or per-tenant RBAC, no secret manager, no high availability, `/metrics` is unauthenticated, the in-process rate limiter is per instance, and the audit chain is not externally anchored.

20.2 Prioritised roadmap (recommendations, not existing features)

1. **Run a scored evaluation of live LLM RCA** against the same tools, with injected adversarial log lines. Rationale: the largest unverified claim.
2. **Replace the simulator's role in validation** by replaying a recorded, anonymised telemetry sample, and measure detector precision and recall.
3. **Move to asynchronous pipeline and agent runs** (a job queue) so requests return immediately.
4. **Add seasonal baselines** and a model-based detector beside the interpretable ones, keeping explanations.
5. **OIDC sign-in, TLS, secret manager, anchored audit head.**
6. **Patch generation beyond reverts**, validated in the sandbox, and real CI integration.
7. **OpenTelemetry traces** across API, RCA and agent.

21 Advanced Technical Deep Dives

Each section says what is implemented and what is a production recommendation.

21.1 C and Python ABI and memory ownership

Implemented: struct-size handshake, per-field `offsetof` tests, out-parameter ownership with matching `free_*`, `try/finally` release, overflow-checked arithmetic, argument range checks before crossing the boundary, allocation-failure injection. *Recommendation:* fuzz the CSV parser (for example libFuzzer) to reach inputs the unit tests do not; run the Python suite against the sanitizer build in CI (the Makefile target exists).

21.2 Statistical detector design and trade-offs

The design chooses *interpretability and low false positives* over sensitivity. Requiring agreement of two detectors, a practical-deviation gate and baseline freezing trade some detection delay for a measured 0.27 false episodes per 100 series-days (recorded, synthetic). The cost is visible in the seeded run: memory is flagged at 14:13, eight minutes after the 14:05 deployment, which is consistent with the memory rule's practical deviation of 6 points while the leak grows about 0.8 points per minute. *Recommendation:* validate on real data before trusting any number.

21.3 Evidence provenance and hypothesis evaluation

Every fact in an RCA carries a tool-call id; the validator refuses uncited or phantom citations; the persisted `tool_calls` hold arguments and results, so each statement can be traced to a database query. The correlation hypotheses carry a score decomposition (temporal, mechanism, topology), which Figure 15 shows.

21.4 Structured LLM tool execution

The engine is the policy enforcement point: the model proposes, the engine validates and runs. Two layers of schema validation, bounded steps, bounded results and read-only incident-scoped access limit what a misled model can do to the cost of a wrong answer, which the citation check and human review then catch or not.

21.5 Policy enforcement and threat modelling

Assets: production deployments, repositories, the audit log, tokens. *Threats considered in code:* an LLM or attacker-influenced log steering actions (the agent ignores LLM text); a user approving their own action (blocked); an API caller bypassing the UI (policy re-checked in the service); a command escaping the sandbox (allowlist, `shell=False`, confinement); log or report injection into HTML (escaping, CSP, sandboxed iframe); brute force and flooding (rate limit, body limit). *Not covered:* a compromised host, a malicious database administrator, supply-chain attacks on dependencies (the project pins versions but does not verify signatures here).

21.6 Database integrity

Foreign keys with cascades, unique audit hashes, run-scoped ids, one migration verified on two engines (recorded), and JSONB reserved for documents. An open design point is `agent_runs.incident_id`, a plain column without a foreign key, which the schema permits to dangle.

21.7 Audit-chain verification

Verification is linear in the number of rows and needs only the stored columns. The dashboard's "Verify chain" requires admin. As noted, the scheme detects accidental or naive tampering but not a rewrite of the whole suffix.

21.8 Integration testing

`test_e2e.py` runs the data-to-report chain and the agent chain through the real API; `test_offline_demo_runs_end_to_end` runs `scripts/demo.py`. These tests catch interface drift between layers that unit tests miss.

21.9 Observability

Implemented: JSON logs with request, incident and agent-run ids; Prometheus metrics (`aegis_http_requests_total`, `aegis_http_request_duration_seconds`, `aegis_http_errors_total`, `aegis_rate_limited_total`, `aegis_agent_actions_total`, `aegis_rca_runs_total`) with bounded label cardinality (route templates). *Not implemented:* traces, dashboards, alerting.

21.10 Performance and scaling considerations

Known: metrics are stored as one row per (cell, metric, minute) (12 x 17 x 360 = 73,440 rows per run), which is simple but will not scale to real fleets; the C parser is linear and the aggregations are `O(n log n)`. Unknown: throughput limits, memory under 10 million records, concurrent users. *Recommendations:* a time-series store, streaming ingestion and batch partitioning, but none of these is built.

22 Glossary and Reference Appendix

22.1 Glossary

Table 19. Terms and acronyms (alphabetical)

Term	Meaning
ABI	Application binary interface: the machine-level layout and calling contract between compiled components
AIOps	Applying data analysis and AI to IT operations
Alembic	Database migration tool for SQLAlchemy
Anomaly	A measurement unusual for its series; in Aegis-Net an episode of flagged points
API	Application programming interface
ASan, UBSan, LSan	AddressSanitizer, UndefinedBehaviorSanitizer, LeakSanitizer: compiler instrumentation that detects memory errors, undefined behaviour and leaks
Audit chain	Rows whose hashes include the previous row's hash, so edits are detectable
Baseline	What a series normally looks like; the reference for deviations
CDR	Call-detail record
Cell	A radio site serving phones
Chain consistency	Share of effects preceded by an observed upstream cause in the causal graph
Container	A running isolated process group started from an image
Correlation	Statistical or temporal association; not causation
ctypes	Python's standard foreign-function library
DevSecOps	Development, security and operations practised together
DEMO-CVE	Synthetic advisory identifier used only in this repository
EWMA	Exponentially weighted moving average
Fault injection	Deliberately making an operation fail to test error handling
Gateway (pgw)	Core-network node routing packet traffic for a group of cells
Hypothesis	A candidate explanation; every Aegis-Net cause is labelled one
IDS	Intrusion detection system
Incident	A correlated group of symptoms needing attention
Jitter	Variation in packet delay
KPI	Key performance indicator
LLM	Large language model
MAD	Median absolute deviation
Mock provider	Deterministic, rule-based stand-in for an LLM
NOC	Network operations centre
Pydantic	Python data-validation library
RBAC	Role-based access control
RCA	Root-cause analysis
RSRP, RSRQ, RSSI, SINR	Radio signal power, quality, strength and signal-to-interference-plus-noise ratio
SRE	Site reliability engineering
Robust z-score	Deviation from the median in units of 1.4826 times MAD
z-score	Deviation from the mean in standard deviations

22.2 Important formulas

Table 20. Formulas and meanings

Formula	Meaning
`drop_rate = dropped / (completed + dropped)`	Drops among established calls
`z = (x - median) / max(1.4826 × MAD, floor)`	Robust z-score (C library)
`z = oriented(x - mean) / max(std, floor)`	Ordinary z-score (Python detector)
`score = 1 - 2^(-z / threshold)`	Anomaly score, 0.5 at the threshold
`alpha = 1 - exp(-dt / tau)`	Time-aware EWMA smoothing factor
`cause score = 0.45 temporal + 0.35 mechanism + 0.20 topology`	Correlation ranking
`temporal = exp(-lead / 1800 s)`	Decay with time since a preceding event
`confidence = top × (0.6 + 0.4 consistency) × (0.9 if close)`, cap 0.95	Incident confidence
`health = max(0, 100 - 30/20/10/5 per incident - 2 per degraded cell)`	Dashboard health score
`queueing_ms = min(400, 4 u / (1 - u))`	Simulator queueing delay

22.3 Repository map

Table 21. Where things live

Path	Contents
`PROJECT_SPEC.md`, `IMPLEMENTATION_PLAN.md`, `ARCHITECTURE.md`, `CLAUDE.md`, `README.md`	Specification, plan with verification log, architecture, engineering instructions, overview
`c-library/`	`include/call_analyzer.h`, `src/*.c`, `tests/`, `CMakeLists.txt`
`backend/call_analyzer/`	ctypes binding
`backend/app/`	`simulation/`, `analytics/`, `models/`, `schemas/`, `api/`, `security/`, `observability/`, `llm/`, `rca/`, `reporting/`, `agents/`, `services/`
`backend/alembic/`, `backend/tests/`	Migration, 12 test modules
`frontend/`	Dashboard source, Dockerfile, nginx config
`scripts/`	Demo, data generators, token and env helpers, smoke and container tests; `scripts/documentation/` regenerates this handbook
`demo-workspace/`	Fictional service copied into agent sandboxes
`docs/`	Topic guides (`agent.md`, `api.md`, `architecture.md`, `rca.md`, `security.md`), `source/aegis.docx` screenshots
`deliverables/`	This handbook, the deck, assets and documentation

22.4 Component and interface reference

Table 22. Interfaces between components

Interface	Provider	Consumer	Contract
C header	`libcallanalyzer`	`call_analyzer` binding	`call_analyzer.h`, `CaStructSizes` handshake
`analyze_call_log()`	binding	pipeline, RCA tools	`CallAnalysisResult` dict
`detect_anomalies()`	analytics	pipeline	`MetricAnomaly` list
`correlate()`	analytics	pipeline	`Incident` objects
`LLMProvider`	`llm/`	RCA engine, reports	`generate`, `tool_call`
`TOOLS` registry	`rca/tools.py`	RCA engine	strict JSON schema plus Pydantic

Interface	Provider	Consumer	Contract
`RCAAnswer`	`rca/schemas.py`	engine, API, reports	validated model
`evaluate()` / `execute_action()`	`agents/`	API	policy decision, state machine
REST API	`backend/app/api/`	dashboard, scripts	OpenAPI

22.5 Environment variables

Table 23. Environment variables (names only; defaults from `config.py`)

Variable	Default	Meaning
`DATABASE_URL`	SQLite under `data/`	SQLAlchemy URL
`AEGIS_DATA_DIR`	`data/`	Generated data directory
`AEGIS_API_TOKENS`	none	`name:role:sha256` entries
`LLM_PROVIDER`	`mock`	`mock`, `anthropic`, `openai`, `local`
`MODEL_NAME`	empty	Model id (required for openai and local; Anthropic has a default)
`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `LOCAL_LLM_BASE_URL`	empty	Provider credentials or endpoint
`REDIS_URL`	empty	Rate-limit store; in-process fallback
`AEGIS_RATE_LIMIT_PER_MINUTE`	120 (compose 600)	Requests per window
`AEGIS_MAX_REQUEST_BYTES`	1,000,000	Body limit
`AGENT_ALLOW_HIGH_RISK`	false	Allow HIGH_RISK actions (still need approval and admin)
`AGENT_WORKSPACE_ROOT`	`/tmp/aegis-agent-workspaces`	Sandbox root
`LOG_LEVEL`	INFO	Logging level
`CALL_ANALYZER_LIB`	unset	Absolute path of the shared library

22.6 Terminal commands

Table 24. Most useful commands

Command	Purpose
`make c-build`, `make c-test`, `make c-test-asan`	Build and test the C library
`make py-venv`, `make py-test`, `make py-test-asan`, `make py-typecheck`	Python environment, tests, strict typing
`make call-logs SEED=42`	Write a seeded CDR file and manifest
`make network-data`	Write simulated telemetry, events and ground truth to `data/`
`make demo`	Offline end-to-end demo
`make env`, `docker compose up --build`, `make docker-smoke`, `make docker-down`	Docker workflow
`make frontend-test`, `make container-test`, `make test`, `make clean`	Dashboard, Python 3.12 container, everything, cleanup
`scripts/documentation/build_all.sh`	Regenerate this handbook and the deck

22.7 Most useful source files and tests

Start with `c-library/include/call_analyzer.h`, `backend/call_analyzer/_native.py`, `backend/app/simulation/model.py`, `backend/app/analytics/detectors.py`, `correlation.py`, `backend/app/rca/engine.py`, `schemas.py`, `backend/app/agents/policy.py`, `service.py`, `runner.py`, `backend/app/security/audit.py`. Matching tests: `c-library/tests/test_call_analyzer.c`, `test_binding_abi_and_ownership.py`, `test_simulation.py`, `test_analytics.py`, `test_correlation.py`, `test_rca.py`, `test_agent.py`, `test_security.py`, `test_e2e.py`.

Appendix A: Screenshot Inventory

Table 25. Screenshots extracted from `docs/source/aegis.docx` into `deliverables/assets/screenshots/` (13 images, all present; none shows a token or secret)

File	Source position	Shows	Used in
`shot_01_src-image11.png` (1917 x 939)	first image	Dashboard, six-question layout	Ch. 16, slide 15
`shot_02_src-image8.png` (1919 x 784)	2nd	Network Overview: 8 KPI cards, 12-cell latency chart	Ch. 7, slide 7
`shot_03_src-image6.png` (1919 x 962)	3rd	Call Analytics with C-library anomalies	Ch. 5, slide 6
`shot_04_src-image5.png` (1897 x 753)	4th	Anomalies table (critical filter)	Ch. 8, slide 8
`shot_05_src-image12.png` (1919 x 562)	5th	Incidents list	Ch. 9, slide 10
`shot_06_src-image9.png` (1897 x 988)	6th	RCA page, evidence and hypotheses (bottom edge cut off in the source)	Ch. 13, slide 11
`shot_07_src-image13.png` (1918 x 881)	7th	AI Reports	Ch. 14, slide 12
`shot_08_src-image7.png` (1899 x 906)	8th	DevSecOps Agent (bottom edge cut off in the source)	Ch. 15, slide 13
`shot_09_src-image3.png` (1007 x 197)	9th	Separation-of-duties message	Ch. 11, slide 14
`shot_10_src-image4.png` (1696 x 323)	10th	Security findings	Ch. 15
`shot_11_src-image10.png` (1688 x 849)	11th	Audit log, chain valid (38 rows) (bottom edge cut off)	Ch. 11, slide 14
`shot_12_src-image1.png` (1689 x 268)	after the label "Viewer:"	Audit page, role error for a viewer	not embedded (same message as shot 13)
`shot_13_src-image2.png` (1670 x 178)	after the label "Viewer:"	Verification unavailable for a viewer	Ch. 11

Note. The source document has one text label, "Viewer:", so the last two images are viewer-role views. The audit log image shows actor names such as `local-admin` and short request ids; no token, key or password appears in any image. Images were embedded at their original aspect ratio (no stretching); text in the wide dashboard captures is legible when zoomed but small at page width.

Appendix B: Verification Record

Table 26. Commands executed for this handbook (working directory: repository root)

Command	Result
`make c-test`	2 of 2 tests passed; exit 0
`make c-test-asan`	2 of 2 tests passed; exit 0
`make py-test`	352 passed, 3 skipped in 207.68 s; exit 0
`make py-typecheck`	"Success: no issues found in 86 source files"; exit 0
`make demo`	`DEMO COMPLETED`; exit 0; incident resolved; audit chain valid (24 rows)
`make frontend-test`	tsc and build succeeded; Vitest 10 of 10 passed; exit 0
`backend/.venv/bin/python scripts/documentation/extract_evidence.py`	One incident, CRITICAL, confidence 0.8947, 36 anomaly episodes

Not executed in this session: `make py-test-asan`, `make container-test`, `make docker-smoke` (Docker stack), PostgreSQL, live Redis and live LLM tests. Their recorded results come from `IMPLEMENTATION_PLAN.md` and are labelled as such.

Appendix C: References

All repository claims were checked against the repository. External sources consulted while writing (retrieved 2026-10-20):

- [R1] Python Software Foundation. *ctypes: A foreign function library for Python.* https://docs.python.org/3/library/ctypes.html (argtypes, restype, structures, pointers).
- [R2] NIST/SEMATECH. *e-Handbook of Statistical Methods: EWMA Control Charts.* https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc324.htm (EWMA definition).
- [R3] FastAPI documentation. *Security.* https://fastapi.tiangolo.com/tutorial/security/ (the `fastapi.security` utilities).
- [R4] NIST/SEMATECH. *e-Handbook: Detection of Outliers.* https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm (modified z-score with MAD, 0.6745 and 3.5).
- [R5] Docker documentation. *Multi-stage builds.* https://docs.docker.com/build/building/multi-stage/.

No claim that Aegis-Net conforms to a named standard is made; where the code resembles a published method, the text says "resembles" and names the differences.